University of Warsaw
Faculty of Economic Sciences

Krzysztof Wojdalski

Nr albumu: 310284

Microstructure-Aware Reinforcement Learning for High-Frequency Trading

Praca magisterska
na kierunku ECONOMICS

The thesis written under the supervision of
dr Pawel Sakowski

April 24, 2026 12:20 UTC

Data Podpis opiekuna

Statement of the Supervisor on Submission of the Thesis

I hereby certify that the thesis submitted has been prepared under my supervision and I declare that it satisfies the requirements of submission in the proceedings for the award of a degree.

Date Signature of the Supervisor:

Oświadczenie autora (autorów) pracy

Data Podpis autora (autorów) pracy

Statement of the Author(s) on Submission of the Thesis

Aware of legal liability I certify that the thesis submitted has been prepared by myself and does not include information gathered contrary to the law.

I also declare that the thesis submitted has not been the subject of proceedings resulting in the award of a university degree.

Furthermore I certify that the submitted version of the thesis is identical with its attached electronic version.

Date Signature of the Author(s) of the thesis

Table of contents

1. Introduction [VERIFIED] 8
1.1. Scope and Objectives of the Research [VERIFIED] . . . 9
1.1.1. Novelty of This Research [VERIFIED] . . . 9
1.1.2. Hypothesis [VERIFIED] . . . 10
1.1.3. Objectives of the Research [VERIFIED] . . . 10
1.2. Thesis Organization and Chapter Overview [VERIFIED] . . . 11

2. Literature Review 13
2.1. Market Microstructure Theory . . . 13
2.2. Market Microstructure Evidence for LOB-Based Trading . . . 16
2.3. Alternative Approaches to Algorithmic Trading . . . 17
2.3.1. Learning Paradigms . . . 17
2.3.2. Decision-System Architectures . . . 18
2.4. Why Reinforcement Learning for Trading . . . 20
2.5. Key Concepts from Reinforcement Learning . . . 21
2.6. RL Applications in Trading Systems . . . 23
2.6.1. Early Applications and Proof-of-Concept . . . 24
2.6.2. From Discrete to Continuous Action Spaces . . . 24
2.6.3. Deep Reinforcement Learning Era . . . 24
2.6.4. Comparative Evidence and Limitations . . . 26
2.6.5. Structured Comparison of Representative Studies . . . 26

3. Reinforcement Learning 29
3.1. Components of a Reinforcement Learning System . . . 29
3.1.1. Environment . . . 29
3.1.2. State . . . 29
3.1.3. Action . . . 30
3.1.4. Policy . . . 30
3.1.5. Reward . . . 30
3.1.6. Value Function . . . 30
3.2. Classifying RL Algorithms . . . 31
3.2.1. Model-free vs Model-based . . . 31
3.2.2. Value-Based, Policy-Based, and Actor-Critic . . . 31
3.2.3. On-Policy vs Off-Policy . . . 32
3.3. Actor-Critic Methods for Continuous Control . . . 32
3.3.1. Value-Based Methods and Their Limitations in Continuous Control . 33
3.3.2. Policy Gradient Methods and the Actor-Critic Architecture . . . 33
3.3.3. DDPG: Deterministic Policies for Continuous Control . . . 34
3.3.4. TD3: Addressing the Instability of Deterministic Actor-Critic Methods 34

4. Design of the Trading Agent 36
4.1. Action Space . . . 36
4.1.1. Continuous vs Discrete Actions . . . 36
4.1.2. Implementation: Box Portfolio Action Space . . . 36
4.1.3. Action Constraints and Transaction Costs . . . 37
4.1.4. Action Smoothing and Exploration Noise . . . 37
4.2. State Space . . . 38

4.2.1. Design Requirements . . . 38
4.2.2. Feature Groups and Selection Rationale . . . 39
4.2.3. Observation Space and Normalization . . . 43
4.2.4. On the Exclusion of Raw LOB Columns . . . 44
4.2.5. Feature Selection Process . . . 45
4.3. Reward Function . . . 45
4.3.1. Why Raw Log-Return Is Insufficient . . . 46
4.3.2. Differential Sharpe Ratio . . . 47
4.3.3. Reward and Financial Return Are Not the Same . . . 48
4.4. Value Function . . . 48
4.4.1. Twin Critics . . . 48
4.4.2. Loss Function . . . 49
4.5. Policy . . . 49
4.6. Discount Factor . . . 49
4.7. Optimization Hyperparameters . . . 49
4.7.1. Learning Rates . . . 49
4.7.2. Weight Decay . . . 50
4.7.3. Scope . . . 50

5. Implementation of the Trading Agent 51
5.1. Simulation Architecture . . . 51
5.2. Simulation Assumptions . . . 51
5.3. Data Preparation . . . 53
5.3.1. Asset Selection . . . 53
5.3.2. Input Format . . . 54
5.3.3. Preprocessing and Splitting . . . 54
5.3.4. Feature Normalization and Leakage Prevention . . . 55
5.3.5. Feature and Price Column Separation . . . 55
5.4. Implementation and Training Pipeline . . . 56
5.4.1. Configuration-Driven Design . . . 56
5.4.2. Feature Pipeline . . . 56
5.4.3. Training Loop . . . 56
5.4.4. Evaluation . . . 58

6. Results [PLACEHOLDER] 59
6.1. Benchmark Strategies [PLACEHOLDER] . . . 59
6.2. Evaluation Protocol [PLACEHOLDER] . . . 60
6.3. Experiment Snapshot [PLACEHOLDER] . . . 60
6.4. Statistical Validation [PLACEHOLDER] . . . 60
6.4.1. Statistical Test Design [PLACEHOLDER] . . . 60
6.4.2. Purpose and Scope of Statistical Validation [PLACEHOLDER] . . . 61
6.4.3. Statistical Significance Summary [PLACEHOLDER] . . . 62
6.4.4. Run-Level Sample Summary [PLACEHOLDER] . . . 63
6.4.5. Descriptive Statistics and Uncertainty Estimates [PLACEHOLDER] . 63
6.4.6. Comparative Validation Strategy for Thesis-Grade Results [PLACEHOLDER] . . . 63
6.4.7. Statistical Risks and Interpretation Caveats [PLACEHOLDER] . . . 64
6.5. Robustness Assessment [PLACEHOLDER] . . . 64
6.5.1. Why Robustness Matters in This Thesis [PLACEHOLDER] . . . 64

6.5.2. Current Evaluation Context [PLACEHOLDER] 65
6.5.3. Robustness Dimensions Considered in This Thesis [PLACEHOLDER] 65
6.5.4. Robustness Matrix for Experimental Reporting [PLACEHOLDER] . . 65
6.5.5. Interpreting Noisy Training Curves in Robustness Terms [PLACEHOLDER] 66
6.5.6. Methodological Limits of the Current Robustness Snapshot [PLACEHOLDER] 66
6.6. Performance Evaluation [PLACEHOLDER] 67
6.6.1. Selected Metrics from the Latest Completed Run [PLACEHOLDER] . 67
6.6.2. Benchmark Comparison [PLACEHOLDER] 67
6.6.3. Evaluation Metadata Snapshot [PLACEHOLDER] 68
6.6.4. Evaluation Plots [PLACEHOLDER] 68
6.6.5. Discussion: Reading Performance in the Context of State Design [PLACEHOLDER] 68

7. Conclusions and Future Work [PLACEHOLDER] 69

Bibliography 71

A. Feature Inventory 75
A.1. LOB Microstructure Features 75
A.2. Bar-Level and Technical Features 78

B. TD3 Hyperparameters 80

C. Experiment Configuration Example 81

D. TD3 Training Workflow Diagram 82

E. TD3 Update Mechanics Diagram 83

F. RL Algorithm Taxonomy 84

G. TD3 Network Architecture 86

H. Data Preparation Pipeline 86
Glossary of Abbreviations and Key Terms 87
H.0.1. Market Microstructure and Trading 87
H.0.2. Reinforcement Learning and Machine Learning 90
H.0.3. Other 93

Abstract

This thesis studies whether a deep reinforcement learning agent using order-book-derived microstructure features can produce robust, risk-adjusted trading performance in a high-frequency single-asset setting. The work focuses on continuous portfolio exposure control and addresses the practical constraints — execution modeling, transaction cost sensitivity, and evaluation discipline — that most often invalidate paper-trading results.

A TD3 agent is trained on high-frequency AAPL limit order book data and evaluated against benchmark strategies under explicit simulation assumptions. The agent achieves positive out-of-sample risk-adjusted returns that exceed the evaluated benchmarks, and a microstructure-aware state representation constructed from order flow, spread regime, and book shape features demonstrates measurable performance improvement over a simpler snapshot-based baseline. Statistical validation and transaction-cost sensitivity analysis are used to separate genuine policy quality from favorable training stochasticity and to bound the conditions under which the observed edge persists.

The main contribution is an end-to-end, reproducible pipeline linking LOB feature engineering, Differential Sharpe Ratio reward design, TD3 training, and rigorous out-of-sample evaluation — together with an explicit account of the simulation assumptions that separate these findings from deployable performance claims.

1. Introduction [VERIFIED]

Much of the reinforcement learning literature in trading relies on low-frequency market representations such as daily or intraday OHLCV bars. High-frequency trading, however, is governed by information embedded in the limit order book, where local liquidity, queue dynamics, spread changes, and event timing determine short-horizon risks. This thesis examines whether a deep RL agent can use order-book-derived features to make favourable trading decisions in a HFT setting.

High-frequency trading should therefore not be treated as a faster version of medium-frequency trading (MFT). The state observed by the agent is different. At short horizons, aggregated price bars omit information that is central to decision-making. Features derived from the order book provide a more detailed description of market conditions than candlesticks. These include depth imbalance, spread variation, order-flow pressure, and inter-event timing. A primary concern of this thesis is whether such microstructure-aware state representations improve the suitability of RL for this domain.

Reinforcement learning is a plausible modelling framework for this problem because trading is sequential and path dependent. Each action affects future exposure, portfolio evolution, and the set of subsequent decisions available to the agent. In high-frequency settings, this distinction matters.

Reinforcement learning has already been deployed in industry settings on limit order book data, but these applications typically address narrower problems such as optimal execution.[1]

Directional trading is a harder problem for several reasons. First, the reward signal at tick frequency is dominated by noise rather than price movement. Second, the LOB state must be transformed into features that carry predictive signal rather than merely describing liquidity conditions. Third, the agent must generalize across intraday regimes whose statistical properties can differ substantially from the training window.

This thesis investigates the use of RL, with particular focus on TD3, for high-frequency single-asset trading based on order-book-derived information. The research is motivated not by a claim that artificial intelligence can universally solve trading, but by a narrower and

[1] An illustrative example is J.P. Morgan's LOXM system, described in contemporaneous reporting as an AI-driven execution tool for client orders rather than a stand-alone directional trading strategy (Noonan 2017). In that execution setting, the limit order book is used to inform decisions about execution quality, fill likelihood, and spread-cost trade-offs, not to forecast short-horizon price direction.

testable question: under controlled experimental conditions, can a continuous control agent using microstructure-aware features achieve superior out-of-sample performance relative to selected benchmarks?

Further constraints arise on both the execution and evaluation sides. Transaction costs can erase apparent profitability, and short-horizon strategies are highly sensitive to turnover, feature specification, and market frictions. Favorable in-sample results may additionally reflect data leakage, overfitting, or unrealistic execution assumptions rather than genuine decision quality.

Accordingly, the thesis focuses on both methodological discipline and model development. It treats feature engineering, chronological evaluation, reward-return separation, and robustness analysis as core parts of the research problem. In this setting, the question is not only whether an agent can learn, but whether the learning setup is properly specified to support credible conclusions.

At the same time, the thesis does not claim to reproduce the full complexity of live high-frequency trading. The framework is based on a controlled simulation environment rather than a complete execution engine with queue-priority modelling, latency competition, or full market impact. For instance, portfolio valuation relies on a mid-price-like execution proxy, and the agent is treated as a price-taking participant that does not alter the state of the order book. The contribution of the thesis should therefore be methodological and empirical: it evaluates RL agents in a microstructure-aware setting while making the simplifying assumptions explicit.

1.1. Scope and Objectives of the Research [VERIFIED]

1.1.1. Novelty of This Research [VERIFIED]

The primary novelty of this thesis is the application of **TD3** to **high-frequency trading with order-book-derived data** for continuous portfolio-exposure control. This differs from a large share of prior RL trading studies that focus on lower-frequency inputs and evaluate algorithms such as Q-learning, DQN, or PPO primarily in bar-level settings.

This thesis contributes by evaluating the RL trading problem in a setting where algorithm class, data granularity, and feature design are jointly aligned with high-frequency market microstructure. The targeted algorithm-data-frequency combination remains less represented

in prior work and defines the core novelty claim tested in the empirical chapters.

1.1.2. Hypothesis [VERIFIED]

The research is based on the following hypotheses:

1. A TD3-based trading agent using order-book-derived high-frequency features can achieve stronger out-of-sample risk-adjusted performance than selected benchmark strategies under the assumptions of the simulated environment.
2. Within the order-book-based trading framework studied here, extending the state representation from basic snapshot features to a broader microstructure-aware feature set leads to improved out-of-sample performance under identical training and evaluation conditions.
3. The empirical performance of reinforcement learning trading agents is materially sensitive to feature specification, reward design, and transaction-cost assumptions, which makes robustness analysis necessary for credible conclusions.

1.1.3. Objectives of the Research [VERIFIED]

The primary research goal is to evaluate RL-based algorithms for high-frequency single-asset trading using order-book-derived data. The research objectives are structured in three interconnected phases:

1. **Design and Implementation**
   - Develop RL trading agents for high-frequency order-book trading applications
   - Construct a testing framework to evaluate agent performance
2. **Testing and Evaluation**
   - Assess agent performance on out-of-sample data against established benchmarks with use of established and novel risk-return metrics
   - Analyze robustness under varying market conditions
   - Determine whether agents can outperform selected benchmark strategies under the defined experimental assumptions
3. **Analysis and Implications**
   - Compare alternative order-book-derived feature sets within a common reinforcement learning training and evaluation framework

- Assess whether broader microstructure-aware feature sets improve out-of-sample results relative to simpler snapshot-based state representations
- Assess the practical relevance and limitations of the approach under simplified execution assumptions
- Identify the binding constraints on out-of-sample performance — whether they originate in execution modeling simplifications, feature set coverage, or training data volume — to prioritize specific extensions for future work

1.2. Thesis Organization and Chapter Overview [VERIFIED]

The thesis proceeds from problem formulation and motivating literature to agent design and implementation, and finally to empirical evaluation.

Chapter 1 introduces the problem context, defines the research scope, and presents the methodological framework used throughout the thesis.

Chapter 2 reviews the literature most directly relevant to the present research question. It covers market microstructure evidence motivating order-book-derived features, the landscape of alternative machine learning paradigms and trading decision architectures, the structural case for reinforcement learning in trading, and prior RL trading applications with attention to action-space design, transaction-cost modeling, and evaluation rigor.

Chapter 3 develops the reinforcement learning foundations required for the thesis, with particular emphasis on the progression from earlier methods to TD3 and on the distinctions that matter for continuous-control trading problems.

Chapter 4 presents the design of the trading agent itself. It defines the continuous action space, the microstructure-aware state representation, the reward formulation, and the main policy and value-function design choices used in the empirical system.

Chapter 5 describes the implementation of the research pipeline, including the simulation architecture, data preparation, configuration-driven feature engineering, code structure, and the TD3-based training workflow used to produce reproducible experiments.

Chapter 6 reports the empirical results. It combines experiment snapshots, performance evaluation, statistical validation, and robustness assessment to separate promising outcomes from noise, overfitting, or fragile conclusions.

Chapter 7 synthesizes the findings, discusses the main limitations of the current framework, and draws implications for future research and practical trading-system development.

2. Literature Review

This chapter surveys the academic foundations relevant to reinforcement learning for high-frequency trading. It is organized in two parts. The first part reviews market microstructure — the theoretical mechanisms and empirical evidence that govern short-horizon price formation in limit order book markets. This establishes the financial environment in which the agent operates and motivates the feature engineering choices made in Chapter 4. The second part positions reinforcement learning as the appropriate framework for this problem. It then surveys the key concepts that matter in a trading context and reviews prior RL trading applications, with attention to action-space design, transaction-cost modeling, and evaluation rigor. The survey concludes by identifying the specific gap — the joint alignment of LOB-derived features, continuous control, and state design — that this thesis addresses.

2.1. Market Microstructure Theory

Market microstructure is grounded in a theoretical literature that models the strategic interactions between informed traders, uninformed liquidity demanders, and market makers in a limit order book. This section reviews the theoretical frameworks that directly underpin the feature design choices made in Chapter 4.

Adverse selection and the spread decomposition. Glosten and Milgrom (1985) established that the bid-ask spread reflects a market maker's protection against trading with informed counterparties. In their model, the market maker faces two types of order flow: uninformed liquidity traders, whose trades are uncorrelated with future price changes, and informed traders, who possess private information about fundamental value. Setting a positive spread allows the market maker to profit from uninformed flow and recoup losses on informed flow. The spread therefore decomposes into an adverse selection component — compensation for information risk — and an inventory or order processing component.

This decomposition has a direct implication for trading signal design. A narrowing spread signals reduced adverse selection risk; a widening spread signals elevated informed activity. These measures, operationalized in Chapter 4 as spread in basis points and the spread ratio, are motivated by this theoretical framework. Their predictive content derives from what the spread reveals about the current composition of order flow, not merely from the mechanical cost of execution.

Kyle (1985) formalized the price impact of informed trading through the lambda parameter, which measures the sensitivity of price changes to order flow. In Kyle's continuous-time model, the informed trader optimally splits their order to minimize price impact, and the market maker sets prices to break even against order flow. The resulting relationship between order imbalance and price change is linear, with slope λ that depends on the ratio of informed to uninformed volume. The order flow features in Chapter 4 — OFI and signed trade flow — can be understood as empirical proxies for Kyle's order imbalance variable, with the bid-ask slope features measuring the depth analogue of λ across levels of the order book.

Limit order book mechanics. Glosten (1994) developed the theory of the open limit order book as a competitive equilibrium, showing that a full LOB can sustain itself as a market mechanism under adverse selection. The key insight is that the price schedule embedded in the LOB — the sequence of bid and ask quotes at increasing depths — reflects the cumulative information content of orders at each quantity level. Deeper levels of the book attract less informed order flow but carry higher execution uncertainty for those posting limit orders.

This theoretical result motivates the multi-level features used in Chapter 4. The book is not a single price but a price schedule, and the shape of that schedule — measured by depth ratios, slopes, and convexity features — encodes information about how informed the marginal liquidity provider expects the market to be at each quantity level.

Order flow and short-term price prediction. Cont, Kukanov, and Stoikov (2014) provide the most direct empirical and theoretical treatment of the relationship between LOB events and short-term price changes. Their framework decomposes all LOB events into limit order arrivals, cancellations, and market orders, and defines order flow imbalance (OFI) as the net directional pressure these events exert on the best bid and ask queues. In a sample of NYSE stocks, they find a linear relationship between OFI and mid-price changes over intervals of one second to one minute, with OFI explaining 60–80% of short-term price variation. Importantly, the relationship is contemporaneous: OFI at time t is correlated with price changes at time t, not a lagged effect, which is consistent with the efficient incorporation of order book information into prices.[2]

[2]The contemporaneous nature of the relationship means OFI does not provide a predictive edge in the strict sense — it reflects information already embedded in prices at the same instant. Its value as a state feature is that it summarises the queue-pressure signal the agent observes at decision time. In practice, the rolling OFI sum in the state vector captures a lagged window of past order-flow pressure, which does carry predictive properties at sub-second horizons.

The theoretical channel is straightforward: when buy-side pressure dominates (positive OFI), market makers raise quotes to attract sell-side flow and reduce inventory risk, mechanically pushing the mid-price upward. The OFI feature in the feature design operationalizes this mechanism directly.

Fair value estimation from the order book. Stoikov (2018) derives the microprice as the theoretically optimal estimate of the next transaction price given the current state of the top-of-book queues. The key observation is that the mid-price — the arithmetic average of best bid and ask — is an inadequate fair value estimate when the queues are imbalanced, because the side with the thinner queue is more likely to be depleted first by incoming market orders, pulling the next transaction price toward the opposite side. The microprice corrects for this by volume-weighting the best quotes:

$$\text{Microprice} = \frac{V_0^{ask} \cdot P_0^{bid} + V_0^{bid} \cdot P_0^{ask}}{V_0^{bid} + V_0^{ask}}$$

When $V_0^{bid} \gg V_0^{ask}$, the microprice is pulled toward the ask — the side more likely to move. The microprice divergence from mid captures this adjustment as a stationary signal and serves as the primary fair value feature in Chapter 4.

Together these four theoretical frameworks establish the informational structure of the limit order book. The spread encodes adverse selection risk (Glosten and Milgrom 1985); price impact and order imbalance are related through Kyle's lambda (Kyle 1985); the shape of the full book encodes the information content at each depth level (Glosten 1994); and OFI is the dominant contemporaneous predictor of short-term price changes (Cont, Kukanov, and Stoikov 2014). The microprice (Stoikov 2018) provides a theoretically optimal fair value estimate that conditions on the current queue imbalance.

These mechanisms are not independent: they all reflect the same underlying process of information aggregation through order flow. The feature set in Chapter 4 is designed to measure each mechanism from a distinct angle, providing the agent with a comprehensive description of the information environment it operates in.

2.2. Market Microstructure Evidence for LOB-Based Trading

The theoretical mechanisms discussed in the preceding section manifest in observable empirical regularities at intraday and tick-level timescales.

At intraday and tick-level timescales, financial markets exhibit structural patterns that are distinct from the macro-level anomalies studied in the asset pricing literature. These patterns arise not from investor irrationality or information asymmetry in the fundamental sense, but from the mechanics of how orders interact in a limit order book: the strategic behavior of market makers, the clustering of institutional activity, and the discrete price grid imposed by exchange rules. Understanding these patterns is a prerequisite for designing an HFT trading agent, since they define the environment in which the agent operates and motivate the feature set used here.

Roll (1984) demonstrated that bid-ask spreads induce negative serial correlation in transaction prices at very short horizons, as they bounce between bid and ask quotes. This microstructure noise distinguishes observed transaction prices from the latent efficient price and creates a predictable pattern exploitable only by agents that operate at tick frequency and account for spread costs explicitly. In this thesis, spread in basis points and the spread ratio feature directly encode the cost of crossing the bid-ask, conditioning the agent's position-sizing decisions on the current liquidity environment.

Wood, McInish, and Ord (1985) and Harris (1986) documented intraday patterns in volume, volatility, and returns: a pronounced U-shape in trading activity and spread width, with elevated liquidity demand at market open and close and a quiet midday period. These patterns reflect institutional trading schedules and liquidity provision dynamics rather than information arrival, and they are stable enough across days to serve as a signal for a trading agent. The hour-of-day sine and cosine encodings in the observation vector are motivated directly by this evidence — the agent needs to distinguish early-session conditions (wide spreads, high adverse selection) from midday periods (tighter spreads, lower activity) when sizing positions.

Harris (1991) showed that prices cluster at round numbers and psychologically attractive levels, creating discontinuities in the order book that affect optimal order placement and execution. In a 10-level LOB, price clustering manifests as uneven spacing between price levels — a pattern captured by the book convexity feature, which measures the curvature

of the price ladder on each side. Awareness of price clustering is also relevant to the spread ratio feature, since clusters create temporary spread compression and expansion that deviates from the recent average.

These three findings collectively establish that short-horizon price dynamics are structured, not random. The bid-ask bounce motivates spread-aware features and reward design; intraday periodicity motivates temporal context features; price discreteness motivates curvature features of the order book. Rather than treating the limit order book as a transparent window onto an efficient price, the empirical evidence supports modeling it as a strategic environment with predictable mechanical regularities — which is precisely the setting in which a reinforcement learning agent can learn to act systematically.

2.3. Alternative Approaches to Algorithmic Trading

Before positioning reinforcement learning as the methodology of choice, it is useful to characterize the main alternatives. Two complementary perspectives are relevant: what machine learning paradigm to use for training, and how the trading decision system is structured architecturally. From the learning-paradigm perspective, the relevant categories are supervised, unsupervised, semi-supervised, and reinforcement learning; the present section discusses the first three explicitly and then motivates reinforcement learning through the architectural comparison that follows.

2.3.1. Learning Paradigms

2.3.1.1. Supervised Learning

Supervised learning trains a model on labeled input-output pairs $(X_i, Y_i)_{i=1}^n$ to minimize a loss function $\mathcal{L}(f_\theta(X), Y)$ over a held-out sample. In trading, supervised approaches have been widely applied to return prediction, directional classification, and volatility forecasting — tasks where historical outcomes provide a natural training signal (Krauss, Do, and Huck 2017; Z. Zhang, Zohren, and Roberts 2019).

The approach has a structural limitation in trading contexts: optimizing the forecast objective and the trading decision are two separate steps with potentially misaligned objectives. A model that minimizes mean squared error on next-period returns may not produce positions that maximize risk-adjusted return after transaction costs, because predictive accuracy and

trading profitability are not equivalent criteria (Moody and Saffell 1998). The separate optimization of prediction and action also discards information: a scalar return forecast does not carry the uncertainty, timing, or context available in the original feature vector.

2.3.1.2. Unsupervised Learning

Unsupervised learning identifies structure in unlabeled data $\{X_i\}_{i=1}^n$ without a target variable. In trading research it is primarily used for market regime detection (clustering volatility or microstructure states), dimensionality reduction of large feature sets (PCA, autoencoders), and anomaly detection in order flow data.

These applications are useful as preprocessing or exploratory tools, but unsupervised learning does not produce trading decisions directly — it carries no reward signal and cannot optimize for a financial objective such as risk-adjusted return. In the present thesis, the regime-conditioning problem is addressed implicitly through engineered microstructure features — spread ratio, inter-event time, temporal encodings — that condition the agent's observation on the current market state without requiring an explicit unsupervised clustering step.

2.3.1.3. Semi-Supervised Learning

Semi-supervised learning combines a small labeled set with a larger unlabeled set, propagating label information through structural similarity in the data. It is useful when labeling is expensive or subjective — medical annotation being the canonical application.

In financial trading, the approach is rarely adopted because the main bottleneck is not label scarcity but the absence of a stable labeling criterion: what constitutes an optimal action at a given market state depends on subsequent price evolution, which is noisy and non-stationary. Constructing pseudo-labels for unlabeled market observations requires either a strong prior on optimal behavior or a separate forecasting model — at which point the problem has already been reformulated as supervised or reinforcement learning.

The same conclusion follows from a second perspective: the architecture of the decision system itself.

2.3.2. Decision-System Architectures

2.3.2.1. Rule-Based Trading Strategies

Rule-based strategies encode trading decisions as explicit conditional logic: position changes are triggered when observable market quantities cross pre-defined thresholds. Common examples include moving-average crossover systems, momentum breakout rules, and mean-reversion strategies based on Bollinger Bands or RSI. In high-frequency settings, rule-based approaches include market-making strategies that post limit orders at fixed offsets from the mid-price and cancel them when inventory limits are reached.

The appeal of rule-based systems is transparency: the causal chain from market condition to action is explicit, auditable, and reproducible. They also impose no distributional assumptions on the data and can be deployed without a training phase.

The limitation is inflexibility. Rules are defined over a fixed set of observable conditions and optimized for a particular market regime. When the regime changes — for example, when a previously mean-reverting asset enters a trending period — fixed rules either require manual recalibration or continue generating losses until the regime reverts. This brittleness under non-stationarity is the primary motivation for learning-based approaches, which can update their behavior from experience rather than requiring explicit rule revision.

2.3.2.2. Indirect Optimization via Forecasts or Labels

An intermediate class of trading systems learns a surrogate target rather than optimizing trading performance directly. One variant employs supervised learning to generate price forecasts from market inputs and then feeds those forecasts into a separate decision rule. Another variant trains directly on labeled state-action pairs, imitating historical or hand-crafted trades without an explicit forecasting stage. In the latter case, the model learns $f : X \to a$ by minimizing a loss over labeled examples (X_i, a_i^*):

$$\min_{\theta} \sum_{i=1}^{N} \ell(f_\theta(X_i), a_i^*)$$

The limitation is the same in both cases: the optimization target is a surrogate rather than the trading objective itself. Forecast-based systems optimize statistical accuracy and then translate predictions into actions through a separate rules layer, discarding much of the contextual information present in the original state. Label-based systems optimize imitation error against actions derived from historical rules or expert behavior, which embeds a prior about what constitutes optimal behavior and does not guarantee maximizing risk-adjusted

return.

2.3.2.3. Direct Optimization of Trading Performance

Direct optimization removes both the intermediate forecast and the labeled action requirement. The agent observes the market state X_t, selects an action, and receives a scalar reward reflecting trading performance. It then adjusts its behavior based on that reward signal — no labels, no predictions, no proxy objective.

The critical difference from the preceding approaches is that the optimization objective is aligned directly with trading performance: there is no surrogate loss whose relationship to actual returns must be assumed. This alignment is the primary motivation for using deep RL. The specific reward formulation and its design rationale are developed in Chapter 4.

2.4. Why Reinforcement Learning for Trading

Before surveying applications, it is worth establishing why reinforcement learning is a natural fit for algorithmic trading rather than an arbitrary methodological choice. The argument rests on three structural properties of the trading problem.

Unknown market dynamics. Model-based approaches require an explicit transition model — a specification of how the environment responds to actions. In financial markets, this model is unknown and non-stationary: price dynamics depend on the aggregate behavior of all participants, which changes continuously. Model-free RL methods learn directly from experience without requiring a transition model, making them the natural choice for this setting (Sutton and Barto 2018).

Off-policy learning from historical data. Online experimentation in live markets is constrained by capital, risk, and regulatory requirements. The alternative is to learn from historical order book data — a fixed dataset of past transitions. Off-policy algorithms such as TD3 (Fujimoto, Hoof, and Meger 2018) can train on stored experience rather than requiring fresh environment interaction at every update, making them sample-efficient in the data-limited financial setting.

Continuous position sizing. Trading decisions are naturally continuous: the agent should be able to express any degree of conviction through the size of its position, not just a binary buy/sell signal. Value-based methods such as DQN (Mnih et al. 2015) require a discrete

action space and cannot represent fractional positions without reintroducing a grid approximation. Deterministic actor-critic methods operate natively in continuous action spaces and are therefore more appropriate for position-sizing tasks.

Together these three properties point toward a model-free, off-policy, continuous-action algorithm — which is the class to which TD3 belongs. The formal algorithmic properties of TD3 are derived in Chapter 3. The present section is concerned with how the broader RL literature has been applied to trading and what that literature implies for the design choices made in this thesis.

2.5. Key Concepts from Reinforcement Learning

This section identifies the reinforcement learning concepts most important for interpreting the finance literature reviewed in this chapter. Formal definitions, equations, and algorithm taxonomies are presented in Chapter 3; the focus here is on why each concept is specifically consequential for trading problems.

```
#| fig-align: center
#| fig-cap: "The agent-environment interaction loop in reinforcement learning. At each st
\usetikzlibrary{arrows.meta}
\begin{tikzpicture}[
  >=Latex,
  thick,
  box/.style={draw, rounded corners=5pt, minimum width=3.2cm, minimum height=1.1cm, font=
]

\node[box] (agent) at (0, 3)  {Agent};
\node[box] (env)   at (0, 0)  {Environment};

% Right path (action)
\draw[->] (agent.east) -- ++(2.8,0) -- ++(0,-3) -- (env.east);
\node[right, align=left, font=\normalsize] at (2.8, 1.5) {action\\[2pt]$A_t$};

% Left path (state + reward)
\draw[->] (env.west) -- ++(-2.8,0) -- ++(0,3) -- (agent.west);
```

```
\node[left,  align=right, font=\normalsize] at (-2.8, 3)   {state\\[2pt]$S_t$};
\node[right, align=left,  font=\normalsize] at (-2.8, 1.5) {reward\\[2pt]$R_t$};

% Dashed line with next-step labels
\draw[dashed, thin] (-1.0, 0.55) -- (-1.0, -0.55);
\node[right, font=\small] at (-1.0, 0.45) {$R_{t+1}$};
\node[right, font=\small] at (-1.0, 0.15) {$S_{t+1}$};

\end{tikzpicture}
```

Portfolio decisions are sequential and path-dependent. This makes the MDP framework (Bellman 1957; Puterman 1994) the formal language for trading agents. However, financial environments only approximately satisfy standard MDP assumptions. Market regime shifts, structural breaks, and partial observability all contribute to this approximation.

Bellman's principle of optimality motivates value-based reasoning about long-run consequences of trading actions. But direct dynamic programming is inapplicable because market transition dynamics are unknown and non-stationary. This motivates model-free learning that estimates value directly from sampled experience (Sutton 1988). Temporal difference methods are especially relevant in finance. Q-learning in particular (Watkins 1989) has an off-policy nature. This permits learning from historical trajectories rather than requiring live interaction with the market.

Portfolio problems are naturally continuous. This makes tabular and discrete-action value methods structurally inadequate. Policy gradient methods (Williams 1992; Sutton et al. 2000) and the actor-critic architectures they support are the class for continuous position sizing. The high-dimensionality of financial state representations makes function approximation necessary. Deep RL methods (Mnih et al. 2015) provide the required generalization capacity. However, this comes at the cost of stability issues in off-policy settings (Tsitsiklis and Van Roy 1997).

This fragility motivated the stabilization mechanisms in TD3 (Fujimoto, Hoof, and Meger 2018). It also recurs throughout the applied trading literature.

Four practical complications distinguish financial RL from standard control benchmarks. The

exploration-exploitation tradeoff is more costly in trading than in most settings. Exploratory actions incur immediate transaction costs, while under-exploration prevents discovery of profitable policies.

This tension is amplified by non-stationarity. Strategies that exploit historical regularities can decay abruptly after regime changes. Yet excessive exploration destroys performance through churn and slippage (Sutton and Barto 2018; Moody and Saffell 1998). Reward specification is a central design decision because it determines what the agent actually optimizes. The reward-shaping literature (Ng, Harada, and Russell 1999) is directly relevant to finance. Misspecified rewards lead to metric gaming or tail-risk neglect.

Sharpe-style risk-adjusted objectives provide an alternative. The Differential Sharpe Ratio (Moody and Saffell 1998) aligns optimization with economically meaningful return rather than with noisy single-step price changes. Sample efficiency is unusually important because historical market data is finite. Live experimentation is also constrained by capital and risk limits. Off-policy replay-based methods (Lin 1992) are therefore methodological enablers rather than engineering conveniences. They underpin the algorithm selection in this thesis. Non-stationarity is the dominant reason why classical RL guarantees do not transfer cleanly to financial markets: market dynamics, volatility regimes, and participant behavior evolve continuously, which motivates the emphasis on robust feature engineering, conservative optimization, and strict temporal validation over rolling out-of-sample periods (Deng et al. 2017).

Accordingly, claims that RL "works" in trading are inherently context-dependent: RL methods can produce useful policies when the learning objective, state representation, and validation design are aligned with market frictions and regime uncertainty.

2.6. RL Applications in Trading Systems

The application of reinforcement learning to financial trading has evolved from theoretical proposals to practical implementations across diverse market segments. This section surveys the literature on RL-based trading systems with a focus on methodology, action space design, and evaluation rigor.

To keep conclusions methodologically defensible, this review distinguishes between: (i) proof-of-concept evidence from backtests under simplified assumptions, and (ii) stronger evidence

that includes robust out-of-sample design, realistic cost modeling, and variance reporting across repeated runs.

2.6.1. Early Applications and Proof-of-Concept

Moody and Saffell (1998) presented the first comprehensive application of RL to trading, introducing recurrent reinforcement learning for optimizing trading systems. His approach incorporated four design choices that remain influential: the Differential Sharpe Ratio as a risk-adjusted reward, a recurrent network architecture for temporal state encoding, direct policy optimization rather than a predict-then-trade pipeline, and explicit transaction cost modeling. Moody demonstrated that RL could outperform supervised learning baselines on S&P 500 futures, establishing feasibility of the approach and showing that optimizing risk-adjusted return directly was often superior to predicting intermediate outcomes.

Moody and Saffell (2001) extended this framework to multi-asset portfolios, validating that RL-optimized policies could generate positive risk-adjusted returns after transaction costs — a stringent test of practical viability.

2.6.2. From Discrete to Continuous Action Spaces

Early value-based approaches applied Q-learning with discrete buy/sell/hold actions to equity and futures markets (Neuneier 1998; J. W. Lee 2001). These studies established that RL could learn profitable patterns from historical data, but also identified persistent problems: sparse rewards, delayed credit assignment, and out-of-sample performance degradation. More fundamentally, discrete action spaces force an artificial granularity on position sizing — a coarse grid loses control precision while a fine grid reintroduces the curse of dimensionality.

Gold (2003) and Dempster and Romahi (2002) demonstrated that policy gradient methods with continuous action spaces addressed this limitation directly. Continuous portfolio weights proved more natural for allocation tasks and policy gradient methods achieved better risk-adjusted returns with fewer training episodes, establishing the case for actor-critic approaches in trading.

2.6.3. Deep Reinforcement Learning Era

The introduction of deep function approximators to RL trading extended applicability to high-dimensional state spaces.

Xiong et al. (2018) and Deng et al. (2017) adapted DQN for equity and cryptocurrency trading, demonstrating that convolutional networks could extract useful features from raw price data. However, both studies noted high variance across runs — a characteristic instability of value-based methods under noisy financial rewards.

Jiang, Xu, and Liang (2017) proposed the portfolio-vector-memory (PVM) architecture, combining policy gradients with recurrent networks for cryptocurrency portfolio management. Their fully differentiable end-to-end design incorporated current holdings into the state and penalized transaction costs explicitly, outperforming Markowitz mean-variance allocation in backtests across 12 cryptocurrencies.

Liang et al. (2018) applied DDPG to stock trading and conducted ablation studies showing that: (i) DDPG outperformed DQN for continuous position sizing, (ii) Sharpe-based rewards improved out-of-sample generalization, and (iii) transaction cost modeling was critical for realistic performance assessment.

Majidi, Shamsi, and Marvasti (2024) provided a focused TD3 study on single-asset trading for AMZN and BTC using daily data and a continuous action space in $[-1, 1]$. Their minimal state representation — lagged close-return windows rather than large technical-indicator sets — reported improved return and Sharpe ratio for TD3 versus discrete variants and buy-and-hold, though results were mixed across assets. This study is best interpreted as evidence that continuous action sizing can be beneficial under realistic frictions, not as definitive proof of robust alpha across market regimes.

Kabbani and Duman (2022) proposed a TD3-based framework incorporating account state (cash and holdings), price features, technical indicators, and optional FinBERT-derived news sentiment. Richer state designs improved both return and Sharpe ratio relative to a close-price-only baseline, with a final 10-stock evaluation reporting a Sharpe ratio of 2.68. Methodologically the paper supports the practical value of feature enrichment in continuous-action actor-critic trading, while also illustrating how performance claims depend strongly on evaluation protocol choices.

Wang (2019) and Yang (2020) demonstrated PPO in cryptocurrency and high-frequency trading settings respectively, noting superior training stability relative to DDPG at the cost of lower sample efficiency. Yang (2020) in particular highlighted the scalability challenges of training stable policies on millisecond-resolution LOB data, requiring extensive hyperparam-

eter tuning.

W. Zhang (2020) integrated attention mechanisms with deep RL for stock trading, enabling the agent to focus on relevant features dynamically. The attention weights provided partial interpretability — visualizing which indicators influenced trading decisions at each timestep — partially addressing the black box critique of deep RL systems.

2.6.4. Comparative Evidence and Limitations

Algorithm comparisons. Shen (2021) benchmarked DQN, DDPG, PPO, TD3, and SAC across asset classes, finding that off-policy methods (DDPG, TD3, SAC) learned faster than on-policy alternatives, with TD3 and SAC exhibiting the lowest variance across random seeds. No algorithm dominated across all markets; best choice depended on reward noise and action space dimensionality.

López de Prado (2018) compared deep RL against traditional quantitative strategies (momentum, mean reversion, pairs trading), noting that RL exhibited superior adaptability during regime changes while traditional strategies offered better interpretability. Hybrid approaches combining domain knowledge with RL often outperformed either in isolation.

Overfitting and generalization. Krauss, Do, and Huck (2017) demonstrated that many published RL trading strategies fail to generalize out-of-sample, attributing this to high noise-to-signal ratios in financial data, limited training data relative to model complexity, and non-stationarity that invalidates training-period patterns. The implication is that evaluation methodology — strict temporal splits, multi-seed reporting, realistic cost assumptions — matters as much as algorithm choice.

Reward specification. Cong (2020) argues that reward engineering is critical and often under-examined. Misspecified rewards can lead to metric gaming (optimizing Sharpe by avoiding trades), ignoring tail risks, and misalignment with actual investor preferences. The Differential Sharpe Ratio (Moody and Saffell 1998) addresses some of these concerns by optimizing risk-adjusted return directly, but does not fully resolve the tension between reward simplicity and the richness of real trading objectives.

2.6.5. Structured Comparison of Representative Studies

Study	Market / Task	State Variables	Action Space	Reward / Objective	Main Caveat
Moody & Saffell (1998)	Futures trading	Price-derived signals + recurrent state	Continuous position control	Differential Sharpe Ratio	Limited modern reproducibility standards
Jiang et al. (2017)	Crypto portfolio allocation	Price tensor + previous portfolio weights	Continuous portfolio weights	Commission-aware portfolio growth	Backtest dependence on selected period and universe
Liang et al. (2018)	Stock portfolio management	Market features + macro variables	Continuous (DDPG)	Return with friction-aware setup	Sensitivity to architecture and hyperparameters
Majidi et al. (2024)	Single-asset AMZN/BTC	Close-return sliding window (autoregressive)	Continuous $[-1, 1]$	Log-return with transaction cost	Mixed performance vs. buy-and-hold across assets
Kabbani & Duman (2022)	Multi-asset stock trading	Cash + holdings + close + indicators + sentiment	Continuous (TD3)	Portfolio return / Sharpe-oriented	Sharpe likely optimistic under strict frozen-policy evaluation
Shen et al. (2021)	Cross-algorithm benchmarking	Task-dependent feature sets	Mixed (discrete + continuous)	Comparative return / risk metrics	Result portability depends on data and protocol harmonization

The literature reviewed here converges on several conclusions relevant to the design choices in this thesis. Continuous-action actor-critic methods — particularly DDPG and TD3 — consistently outperform discrete-action alternatives for position-sizing tasks, a finding confirmed across independent studies with different assets and market regimes (Liang et al. 2018; Majidi, Shamsi, and Marvasti 2024; Kabbani and Duman 2022). Risk-adjusted reward signals, especially Differential Sharpe Ratio formulations, tend to improve out-of-sample generalization relative to raw return objectives, though they must be paired with evaluation on realized financial metrics rather than the training signal itself to avoid conflating optimization proxy with economic performance. Explicit transaction cost modeling is a prerequisite for any realistic performance claim at the frequencies considered here.

A consistent gap in the literature is the reliance on OHLCV or candlestick representations as the primary state input. Studies that operate at tick frequency and exploit the full depth of the limit order book remain scarce, despite the theoretical case — established in the microstructure literature — that the informational content relevant to short-horizon price prediction is concentrated precisely in the queue dynamics and imbalance structure that OHLCV aggregates discard. The present thesis addresses this gap directly by constructing the agent's observation vector entirely from 10-level order book data, using features grounded in the same theoretical frameworks reviewed in the preceding section.

3. Reinforcement Learning

This chapter develops the reinforcement learning foundations required for the trading agent implemented here. It is organized in three parts. The first part introduces the core components of an RL system — the formal vocabulary of states, actions, rewards, policies, and value functions that all subsequent algorithms inherit. The second part classifies RL algorithms along the dimensions that matter most for the trading problem: model-free vs. model-based, and on-policy vs. off-policy. The third part traces the algorithmic lineage from early value-based methods to TD3, showing how these properties converge on TD3 as the algorithm of choice for continuous-control trading.

3.1. Components of a Reinforcement Learning System

Reinforcement learning systems address sequential decision-making problems by selecting actions that maximize cumulative discounted future rewards. This subsection defines the core components using trading as the primary illustrative context, drawing on Sutton and Barto (2018).

3.1.1. Environment

The environment defines the space of possible states and the rules governing transitions between them. In the trading setting studied here, the environment is the simulated order-book market: it receives a portfolio-exposure action from the agent at each time step, updates the portfolio value according to price changes and transaction costs, and returns the new state and reward. The environment encapsulates everything outside the agent's direct control — price dynamics, spread, and market impact — under a set of explicit simulation assumptions.

3.1.2. State

The state s_t is a snapshot of the environment at time t that contains sufficient information for the agent to select an action. In the HFT setting of the state is a vector of engineered microstructure features derived from the limit order book: imbalance measures, spread, order flow signals, and temporal encodings. States can be terminal, ending an episode, or non-terminal, continuing the interaction sequence. The Markov property requires that the optimal action depend only on s_t and not on the full history; the feature engineering in Chapter 4 is designed to approximate this property.

3.1.3. Action

The action a_t is the decision the agent takes given state s_t. In this thesis the action space is continuous: $a_t \in [-1, 1]$, representing the target portfolio exposure from maximum short to maximum long. The continuous formulation is important because it allows the agent to express varying degrees of conviction through position size, rather than being restricted to a binary buy/sell decision.

3.1.4. Policy

The policy $\mu_\theta : \mathcal{S} \to \mathcal{A}$ maps states to actions. A deterministic policy is used — given the same state, the policy always produces the same action. This is appropriate for a trading agent expected to behave consistently under identical market conditions. During training, exploration noise is added to the policy output to encourage discovery of profitable behaviors; at evaluation time the policy is used without noise.

3.1.5. Reward

The reward r_t is a scalar signal returned by the environment after the agent takes action a_t in state s_t. It provides the only learning signal available to the agent — there are no labels, no supervisor. The reward function is therefore a critical design choice: it determines what the agent actually optimizes. In trading, rewards are typically derived from portfolio returns, possibly adjusted for risk and transaction costs. The specific reward formulation used here is developed in Chapter 4.

3.1.6. Value Function

The value function $V^\pi(s)$ estimates the expected cumulative discounted reward starting from state s under policy π:

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \,\middle|\, s_t = s \right]$$

where $\gamma \in [0, 1]$ is the discount factor. The value function allows the agent to reason about long-run consequences rather than only immediate rewards — a necessary capability in trading, where the cost of a position change today affects future returns through transaction

costs and portfolio path dependence. The action-value function $Q^\pi(s, a)$ extends this to state-action pairs and forms the basis of the critic networks in actor-critic algorithms.

3.2. Classifying RL Algorithms

3.2.1. Model-free vs Model-based

Reinforcement learning algorithms divide into two broad families depending on whether they require a model of the environment.

Model-based methods learn or are given a transition model $P(s'|s, a)$ and use it to plan ahead — computing value functions by rolling out simulated trajectories rather than purely from observed data. Dyna-Q and AlphaZero are canonical examples. When the model is accurate, planning is data-efficient; when the model is wrong, planning amplifies errors. In financial markets, transition dynamics are unknown and non-stationary, making a reliable model practically unattainable.

Model-free methods learn directly from sampled transitions without building a transition model. They are less sample-efficient but impose no assumptions on the environment's dynamics. TD3 belongs to this family.

3.2.2. Value-Based, Policy-Based, and Actor-Critic

A second dimension classifies algorithms by what they learn and optimize.

Value-based methods learn a value function and derive a policy implicitly by acting greedily with respect to it. Q-learning and its deep variant DQN (Mnih et al. (2015)) are prototypical examples. They work well in discrete action spaces but struggle when actions are continuous, because the greedy step $\arg\max_a Q(s, a)$ becomes intractable over an uncountable set.

Policy-based methods directly parameterize and optimize the policy π_θ. REINFORCE (Williams (1992)) updates parameters by gradient ascent on expected return:

$$\theta \leftarrow \theta + \alpha \nabla_\theta \mathbb{E}\left[\sum_t r_t\right]$$

Policy gradients handle continuous actions naturally but suffer from high variance, requiring many samples to obtain a reliable gradient signal.

Actor-critic methods combine both: an actor μ_θ selects actions and a critic $Q_w(s, a)$ evaluates them, using the critic's lower-variance estimates to guide actor updates. The full actor-critic derivation and TD3's specific extensions are given in Section 3.3.

3.2.3. On-Policy vs Off-Policy

On-policy methods evaluate and improve the same policy used to collect experience. SARSA is a canonical example, updating with:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$$

where a_{t+1} is drawn from the current policy. On-policy methods tend to be stable but wasteful: each sample is used once and then discarded.

Off-policy methods decouple the data-collection policy (the behavior policy) from the policy being improved (the target policy). Q-learning updates with the greedy maximum:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_t + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t)]$$

regardless of how a_{t+1} was actually selected. This separation allows experience from a replay buffer — collected under earlier or exploratory policies — to be reused many times. TD3 is off-policy and uses a replay buffer to reuse collected transitions; the implications for sample efficiency in financial settings are discussed in Section 3.3.

3.3. Actor-Critic Methods for Continuous Control

The development of reinforcement learning algorithms is a series of conceptual breaks , each motivated by a limitation in the preceding framework. This section traces the specific lineage that leads to the Twin Delayed Deep Deterministic Policy Gradient (TD3) algorithm, which is employed here. The survey is not exhaustive; it focuses on the algorithmic properties that are directly relevant to the continuous-action, single-asset trading problem formulated in Chapter 4.

3.3.1. Value-Based Methods and Their Limitations in Continuous Control

The foundational algorithms of the field — Q-learning (Watkins and Dayan 1992) and its on-policy variant SARSA (Rummery and Niranjan 1994) — operate over tabular representations of state-action value functions. Both methods converge to the optimal policy under mild conditions, but only when the state-action space is small enough to enumerate explicitly. In a trading context with continuous price-derived features, the state space is effectively infinite, and a tabular representation is computationally infeasible.

Deep Q-Networks (DQN) (Mnih et al. 2015) resolved the scalability problem by replacing the lookup table with a deep neural network, enabling generalization across states. Two architectural innovations were critical: experience replay, which decorrelates training samples by storing and randomly resampling past transitions, and a periodically frozen target network, which stabilizes the Bellman update by preventing the target from shifting at every gradient step. DQN achieved human-level control across a wide range of tasks, but it inherits a structural constraint from Q-learning: it requires a discrete action space, because the max operator in the Bellman target presupposes that all actions can be enumerated. For a trading agent that outputs a continuous portfolio weight in $[-1, 1]$, this is not an acceptable constraint. Discretizing the action space into a finite grid resolves the representation issue formally, but it reintroduces the granularity problem: a coarse grid loses the benefits of continuous control, while a fine grid makes the maximization step computationally expensive and recovers the curse of dimensionality in higher-dimensional portfolio settings.

3.3.2. Policy Gradient Methods and the Actor-Critic Architecture

A distinct class of algorithms avoids the maximization problem by parameterizing the policy directly and optimizing it through gradient ascent on expected cumulative reward. The foundational result, REINFORCE (Williams 1992), uses Monte Carlo estimates of the policy gradient:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\left[\nabla_\theta \log \pi_\theta(a_t|s_t) \cdot G_t\right]$$

where G_t is the return from time t. While REINFORCE applies naturally to continuous action spaces through a parameterized distribution (for example, a Gaussian policy), it suffers from high variance in the gradient estimate because G_t is computed from a full episode trajectory,

making it sensitive to trajectory length and reward scale.

The actor-critic architecture reduces this variance by replacing G_t with a learned advantage estimate $A(s_t, a_t) = Q(s_t, a_t) - V(s_t)$, maintained by a separate critic network. The actor updates the policy in the direction of higher-valued actions, while the critic provides a lower-variance baseline. This separation is the architectural foundation of all subsequent continuous-action algorithms, including TD3.

3.3.3. DDPG: Deterministic Policies for Continuous Control

Lillicrap et al. (2016) introduced Deep Deterministic Policy Gradient (DDPG), which extends the actor-critic framework to deterministic policies over continuous action spaces. Rather than sampling an action from a distribution, the actor outputs a single action $\mu_\theta(s)$ directly. The policy gradient simplifies to:

$$\nabla_\theta J(\theta) = \mathbb{E}_s \left[\nabla_a Q_\phi(s, a)\big|_{a=\mu_\theta(s)} \cdot \nabla_\theta \mu_\theta(s) \right]$$

DDPG inherits the experience replay and target network mechanisms from DQN, making it an off-policy algorithm and therefore sample-efficient. However, empirical studies consistently report two failure modes in DDPG. First, the critic tends to overestimate action values due to the maximization bias introduced when the target value is computed using the same network that is being updated. Overestimated Q-values propagate into the actor update, pushing the policy toward actions that exploit the critic's errors rather than the true reward signal. Second, the algorithm is sensitive to the relative frequency of actor and critic updates: updating the actor before the critic has converged amplifies the overestimation problem. In financial environments, where rewards are noisy and non-stationary, these instabilities are particularly damaging because the critic cannot distinguish between genuine market signal and approximation artifacts.

3.3.4. TD3: Addressing the Instability of Deterministic Actor-Critic Methods

Fujimoto, Hoof, and Meger (2018) introduced TD3 as a direct response to DDPG's failure modes, proposing three targeted mechanisms.

Clipped double Q-learning. Two independent critic networks Q_{ϕ_1} and Q_{ϕ_2} are maintained, and the Bellman target is constructed using the minimum of their estimates:

$$y = r + \gamma \min_{i=1,2} Q_{\phi_i}(s', \tilde{a}')$$

Taking the minimum provides a conservative estimate of target value and substantially reduces overestimation bias. This is analogous to the double Q-learning correction (Hasselt, Guez, and Silver 2016), adapted to the deterministic actor- critic setting.

Delayed policy updates. The actor and target networks are updated less frequently than the critics — typically once for every two critic updates. This allows the value estimates to stabilize before they are used to compute the policy gradient, breaking the feedback loop between an unstable critic and an erroneous actor update.

Target policy smoothing. When computing Bellman targets, noise is added to the target action $\tilde{a}' = \mu_{\theta'}(s') + \epsilon$, with ϵ clipped to a small interval. This prevents the critics from fitting sharp value peaks caused by function approximation, acting as a regularizer on the value landscape.

Together, these three modifications produce an algorithm that is substantially more stable than DDPG on standard continuous control benchmarks while retaining its off-policy sample efficiency. The combination of stability under noisy rewards, data efficiency through experience replay, and native support for continuous actions makes TD3 well suited to the continuous portfolio allocation problem studied here. Alternative algorithms such as SAC (Haarnoja et al. 2018) incorporate maximum-entropy objectives and stochastic policies, which may improve exploration but introduce stochasticity into the policy at evaluation time — a less natural fit for a trading agent that is expected to produce consistent, interpretable allocation decisions given identical market conditions.

4. Design of the Trading Agent

This chapter applies the reinforcement learning framework introduced in Chapter 3 to single-asset high-frequency trading. Each section addresses one component of the MDP formulation: the action space, state representation, reward function, value function, policy, discount factor, and step size. The choices made here are motivated by the structure and constraints of the trading problem.

4.1. Action Space

The action space defines the set of permissible decisions available to the trading agent at each time step. This design choice fundamentally shapes the agent's behavior and determines the granularity of control over portfolio positions.

4.1.1. Continuous vs Discrete Actions

Trading systems can be formulated with either discrete or continuous action spaces:

Discrete action spaces restrict the agent to a finite set of categorical decisions. The two most common formulations are the three-action set $\{-1, 0, 1\}$ representing short, neutral, and long positions, and the two-action set $\{0, 1\}$ representing only short and long positions without a neutral option.

Continuous action spaces allow the agent to output any value within a specified range, enabling smooth portfolio allocation decisions.

4.1.2. Implementation: Box Portfolio Action Space

In this implementation, the action space is defined as a continuous interval:

$$\mathcal{A} = [-1.0, 1.0]$$

where the action $a_t \in \mathcal{A}$ at time t represents the target portfolio allocation for the traded asset. The semantic interpretation is:

- $a_t = 1.0$: Maximum long exposure (fully invested long position)
- $a_t = 0.0$: Neutral position (no exposure, cash equivalent)
- $a_t = -1.0$: Maximum short exposure (fully invested short position)

- $a_t \in (0, 1)$: Partial long position scaled by magnitude
- $a_t \in (-1, 0)$: Partial short position scaled by magnitude

This formulation explicitly supports short-selling, which is realistic in foreign exchange and cryptocurrency markets where symmetric long/short positions are standard.[3] The continuous nature enables the agent to express varying degrees of conviction—for example, $a_t = 0.3$ indicates a moderately bullish stance, while $a_t = -0.8$ signals strong bearish conviction.

4.1.3. Action Constraints and Transaction Costs

The action space is bounded to prevent unrealistic leverage. In practice, unbounded actions could lead to positions exceeding available capital or margin requirements. The $[-1, 1]$ range implicitly assumes unit leverage without margin multipliers.

Transaction costs are applied proportionally to position changes. If the agent transitions from a_{t-1} to a_t, the turnover magnitude is $|a_t - a_{t-1}|$, and fees are calculated as:

$$\text{fee}_t = \text{trading_fees} \times |a_t - a_{t-1}| \times \text{notional value}$$

This incentivizes the agent to minimize unnecessary portfolio rebalancing, as frequent adjustments erode returns. During training, the RL algorithm must learn to balance the potential gains from reallocation against the friction costs incurred.

4.1.4. Action Smoothing and Exploration Noise

During training, exploration is introduced through additive Gaussian noise:

$$a_t^{\text{noisy}} = \text{clip}(a_t + \epsilon, -1.0, 1.0)$$

where $\epsilon \sim \mathcal{N}(0, \sigma^2)$ and σ is the exploration noise standard deviation.[4] The clipping operation ensures actions remain within valid bounds. This exploration mechanism encourages

[3] In equity markets, short positions incur securities-lending borrow costs that are not captured by the proportional transaction-cost model used here. The simulation treats long and short positions as cost-symmetric, which is the standard simplification in single-asset RL trading environments but represents an additional optimistic assumption beyond the mid-price execution proxy noted elsewhere.

[4] The main experiment uses $\sigma = 0.2$. The range 0.1–0.3 reflects common practice in continuous-action TD3 applications; values above 0.3 tend to destabilize learning in low-signal environments such as tick-frequency returns.

the agent to discover diverse trading strategies during early training phases while gradually converging to deterministic exploitation as learning progresses.

Additionally, TD3 employs target policy smoothing during critic updates, adding noise to target actions when computing Bellman targets. This regularization technique prevents the critics from overfitting to narrow peaks in the value function caused by function approximation errors.

4.2. State Space

The state space defines the informational representation available to the agent at each decision point. At each time step t, the agent receives an observation vector $s_t \in \mathcal{S}$ constructed from the current state of the limit order book (LOB). The design of this observation is the primary engineering decision in the empirical pipeline: the agent can only exploit structure that is present in its input.

4.2.1. Design Requirements

An effective state representation for high-frequency trading must satisfy several criteria simultaneously.

Approximation of the Markov property. The TD3 algorithm assumes that s_t contains sufficient information such that the optimal action depends only on the current state, not on the full history. Raw price levels violate this assumption because their unconditional distribution is non-stationary; a price of 180 means something very different depending on recent trajectory. Engineered features — returns, imbalances, normalized spreads — are designed to be more stationary and to encode the local market state rather than the absolute price level.

Informativeness about short-term price direction. For the reward signal to be learnable, the observation must contain features that are predictive of near-term mid-price changes. The selection is therefore grounded in market microstructure theory, which identifies specific LOB quantities — imbalances, order flow, spread — as the empirically dominant predictors of short-term price movement.

Dimensionality discipline. Each additional observation dimension enlarges the input space that the policy and critic networks must generalize over. Features are included only when

they carry a distinct theoretical mechanism not already captured by other features in the set.

Scope and timescale consistency. The observation is restricted to LOB-derived features. Bar-based signals such as OHLCV aggregates or technical indicators require a fixed aggregation window. This imposes a coarser timescale than the tick-level decisions being made.

For example, a one-second bar collapses dozens of individual order-book events into a single price summary. This discards the queue dynamics, imbalance shifts, and spread fluctuations that constitute the decision-relevant information at this resolution.

Cross-asset signals such as VIX or index returns face a related problem. They update at frequencies far below the inter-event times of the AAPL tick stream. This contributes no new information between consecutive book updates.

Aligning multiple tick streams at sub-second precision also introduces synchronization risk. A VIX observation and an AAPL LOB snapshot sharing a timestamp to the second are not the same market moment.

Beyond these practical constraints, restricting the state to single-asset LOB features is a deliberate experimental choice. The thesis tests whether microstructure-aware features alone are sufficient for a viable agent. Adding cross-asset signals would answer a different question and prevent clean attribution of any observed performance to LOB structure.

A natural extension for future work would be to augment the LOB observation with synchronized regime indicators. These could include intraday VIX futures at the minute level, sector ETF order flow, or options-derived skew.

This would be done once the baseline LOB-only performance is established. It would allow a controlled ablation of which additional signal sources materially improve out-of-sample results.

4.2.2. Feature Groups and Selection Rationale

The state vector is composed of five groups of features, each targeting a distinct aspect of LOB microstructure.

4.2.2.1. Imbalance Features

Volume imbalance at a given price level i is defined as:

$$\text{BookPressure}_i = \frac{V_i^{bid} - V_i^{ask}}{V_i^{bid} + V_i^{ask}}$$

Imbalance signals the relative weight of resting liquidity on each side and has been shown to predict short-term price direction (Cao, Chen, and Griffin 2004; Biais, Hillion, and Spatt 1995). The multi-level aggregate version averages this signal across the top N levels:

$$\text{OBI}_N = \frac{\sum_{i=0}^{N-1}(V_i^{bid} - V_i^{ask})}{\sum_{i=0}^{N-1}(V_i^{bid} + V_i^{ask})}$$

The multi-level formulation is more robust to noise at the best quote and captures the depth structure of the book. Both single-level (levels 0–2) and three-level aggregate variants are included.

Order count imbalance complements the volume-based measures by counting the number of resting orders rather than their aggregate size, making it sensitive to fragmentation that a single large institutional order would mask:

$$\text{OCI}_i = \frac{C_i^{bid} - C_i^{ask}}{C_i^{bid} + C_i^{ask}}$$

4.2.2.2. Fair Value Features

The microprice (Stoikov 2018) is a volume-weighted estimate of where the next transaction price is likely to occur:

$$\text{Microprice} = \frac{V_0^{ask} \cdot P_0^{bid} + V_0^{bid} \cdot P_0^{ask}}{V_0^{bid} + V_0^{ask}}$$

When the ask side is thin relative to the bid side, the microprice is pulled toward the ask — the side more likely to move first. The microprice divergence from mid-price isolates the imbalance component of fair value as a stationary signal:

$$\text{MicroDiv} = \text{Microprice} - \frac{P_0^{bid} + P_0^{ask}}{2}$$

The volume-weighted mid-price skew extends this concept across three levels, measuring

whether deep liquidity is positioned asymmetrically relative to the current mid-price.

4.2.2.3. Spread and Cost Features

The bid-ask spread, expressed in basis points, is the primary observable measure of adverse selection cost (Kyle 1985; Glosten and Milgrom 1985):

$$\text{SpreadBps} = \frac{P_0^{ask} - P_0^{bid}}{\frac{P_0^{bid} + P_0^{ask}}{2}} \times 10{,}000$$

A widening spread signals elevated information asymmetry or inventory risk among market makers. The spread ratio — current bid-ask relative to its recent rolling mean — separates the level effect from the regime signal, flagging periods of abnormally elevated adverse selection risk (Easley, López de Prado, and O'Hara 2012).

Bid and ask slopes measure the price elasticity of each side of the book across five levels (Bouchaud, Mézard, and Potters 2002), capturing market impact per unit of trading volume and serving as a proxy for Kyle's lambda (Kyle 1985):

$$\text{BidSlope} = \frac{P_0^{bid} - P_4^{bid}}{\sum_{i=0}^{4} V_i^{bid}}, \qquad \text{AskSlope} = \frac{P_4^{ask} - P_0^{ask}}{\sum_{i=0}^{4} V_i^{ask}}$$

4.2.2.4. Order Flow Features

Order flow imbalance (OFI) (Cont, Kukanov, and Stoikov 2014) measures the net directional pressure from LOB events between consecutive snapshots, conditioning on whether each event represents genuine queue growth rather than a price improvement:

$$\text{OFI}_t = \Delta V_t^{bid} \cdot \mathbf{1}\left[P_{0,t}^{bid} \geq P_{0,t-1}^{bid}\right] - \Delta V_t^{ask} \cdot \mathbf{1}\left[P_{0,t}^{ask} \leq P_{0,t-1}^{ask}\right]$$

Cont, Kukanov, and Stoikov (2014) establish OFI as the single strongest predictor of short-term price changes in LOB data. A rolling sum over 50 events captures persistent directional pressure over multiple tick horizons.

Signed trade flow (cumulative delta) (C. M. C. Lee and Ready 1991) is constructed directly from the trade tape rather than from book changes, providing a complementary signal that measures executed volume rather than queue dynamics:

$$\text{CumDelta}_{W,t} = \sum_{\tau=t-W+1}^{t} \text{size}_\tau \cdot \mathbf{1}[\text{side}_\tau = B] - \text{size}_\tau \cdot \mathbf{1}[\text{side}_\tau = A]$$

Queue depletion measures how rapidly the best-bid or best-ask queue is being consumed across consecutive snapshots, identifying situations where the top-of-book queue is close to exhaustion and a price move is imminent.

Odd-lot trade ratio and odd-lot imbalance (Boehmer et al. 2021) decompose trade flow by order size. In the AAPL dataset used here, 81% of trades are odd-lot sized (below 100 shares).[5] This primarily reflects retail and algorithmic small-lot activity.

A sudden drop in the odd-lot ratio signals increasing institutional participation. This indicates consequently higher adverse selection risk.

4.2.2.5. Regime and Temporal Features

Two features condition the agent on the current market regime rather than providing directional signals.

Log inter-event time (Engle 2000) captures the activity rate of the market:

$$\text{LogGap}_t = \log\big(1 + (t_{\text{ns}} - t_{\text{ns},t-1})\big)$$

Dense update bursts indicate informed order flow or a large order working through the book; sparse updates signal low activity and reduced adverse selection risk.

Mid-price acceleration is the second finite difference of the mid-price:

$$\text{Accel}_t = M_t - 2M_{t-1} + M_{t-2}, \qquad M_t = \frac{P_0^{bid} + P_0^{ask}}{2}$$

It distinguishes momentum regimes (positive, accelerating) from mean-reversion regimes (negative, decelerating) at tick frequency.

[5] This figure is computed from the February 25–27, 2026 AAPL MBP-10 dataset used in the main experiment and should not be interpreted as a long-run population statistic. The share of odd-lot trades in AAPL has increased substantially since the SEC's 2023 amendments to odd-lot reporting rules and may vary across periods.

The sine and cosine encodings of the hour-of-day capture the well-documented U-shaped intraday pattern in volume and spread (Wood, McInish, and Ord 1985):

$$\text{HourSin}_t = \sin\left(\frac{2\pi \cdot h_t}{24}\right), \qquad \text{HourCos}_t = \cos\left(\frac{2\pi \cdot h_t}{24}\right)$$

The cyclical encoding ensures temporal continuity — 09:30 and 16:00 remain structurally distinct without imposing an arbitrary ordinal distance.

4.2.2.6. Position Feature

The observation vector is optionally extended with the agent's own current portfolio exposure $a_{t-1} \in [-1, 1]$. Unlike the LOB-derived features, this quantity is not computed from market data but read from the simulated broker state at each step — it is a runtime feature appended after the static feature vector is constructed.

Including the current position makes the state partially Markovian with respect to inventory. Without it, the agent cannot distinguish between arriving at exposure 0 from a long position versus arriving from a short position.

This distinction matters because the cost of the next action differs in both cases through the position-change fee. With the position feature, the agent can condition its action on whether a change would incur a fee. It can also learn to penalize unnecessary turnover. The feature is not z-score normalized. Because the position is bounded in $[-1, 1]$ by the action space and is drawn from a bimodal distribution that concentrates near the extremes (fully long or fully short), standardization would distort this natural structure: a z-score transform on a bimodal distribution produces a U-shaped output with negative variance contributions between the modes, which is worse than the raw signal for a neural-network input.[6]

4.2.3. Observation Space and Normalization

The complete observation vector $s_t \in \mathbb{R}^d$ is formed by concatenating all engineered features. The exact dimensionality d is scenario-dependent and specified in the configuration files governing each experiment.

[6]This is a practical consideration rather than a theoretical constraint. If the position distribution were approximately Gaussian, z-score normalization would reduce the scale mismatch with the other features. However, TD3 policies with continuous action spaces in $[-1, 1]$ tend to produce positions that cluster near the boundaries, making the raw representation both simpler and better matched to the actual distribution.

All features except cyclical time encodings and the position feature are standardized using training-set statistics:

$$\tilde{x}_i = \frac{x_i - \mu_i^{\text{train}}}{\sigma_i^{\text{train}}}$$

The mean μ_i^{train} and standard deviation σ_i^{train} are computed exclusively on the training split and applied without modification to the validation and test splits, preventing any form of data leakage. Cyclical encodings are inherently bounded in $[-1, 1]$ and require no normalization. The position feature is likewise not standardized; as discussed in the preceding section, it is already bounded in $[-1, 1]$ and its bimodal distribution makes z-score normalization counterproductive.

The formal observation space is:

$$\mathcal{S} = \mathbb{R}^d$$

In practice, after standardization, feature values concentrate in $[-3, 3]$ under approximate Gaussian assumptions. The policy and value networks must generalize to out-of-distribution observations that arise during stress episodes or regime shifts not present in the training sample.

4.2.4. On the Exclusion of Raw LOB Columns

Raw price and size columns — that is, the unprocessed bid and ask prices and sizes at each level — were deliberately excluded from the observation vector. Prior work in the deep learning tradition feeds the raw LOB matrix directly to the network (Z. Zhang, Zohren, and Roberts 2019), allowing convolutional or recurrent architectures to discover structure from data without hand-engineered intermediaries. This is a coherent approach when the model itself is the feature extractor.

In the present setup, however, comprehensive microstructure features with established theoretical mechanisms are the primary input to the agent. Including raw columns alongside them introduces redundancy without adding an interpretable signal: normalized bid sizes at each level carry substantially the same information as the imbalance features derived from

them, and normalized absolute prices at tick frequency are near-meaningless once the local market state is already encoded through microprice divergence, OFI, and spread measures. Mixing both paradigms would imply that the engineered features are insufficient, which is inconsistent with the selection rationale described above. The observation vector is therefore composed entirely of theoretically motivated derived features.

4.2.5. Feature Selection Process

Feature selection in this thesis follows a theory-driven criterion. Each of the 25 features in the final observation vector was included because it represents a distinct mechanism identified in the microstructure literature: imbalance at multiple depth levels, fair value estimation, spread regime, book shape, order flow, and intraday temporal context. A feature was retained if it contributed information not already captured by other variables in the set. This yields a conservative specification whose priority is interpretability and alignment with theory rather than exhaustive coverage of all computable LOB statistics. Automated ranking or pruning procedures — such as forward selection, LASSO, or mutual information screening — were therefore not part of the selection process.

The selection is operationalized through a registry-driven configuration pipeline. Feature names defined in the scenario YAML file resolve to computation functions in the feature registry at runtime; adding, removing, or reordering features requires only a change to the configuration file, not to source code. The active feature set for the main experiment is defined in `src/configs/features/hft_lob_features_state_space.yaml` and referenced by the scenario file `src/configs/scenarios/aapl_td3_hft_lob_state_space.yaml`.

The final state vector has dimensionality $d = 26$: 25 LOB-derived features across five groups (5 imbalance, 3 fair value, 6 spread and cost, 7 order flow, 4 regime and temporal) plus the current portfolio exposure appended at runtime. All features except the cyclical hour encodings are normalized using training-split statistics as described in the preceding section. The complete feature specification with parameter values is listed in Appendix A.

4.3. Reward Function

The reward function determines what the agent actually optimizes. The thesis implements direct optimization of trading performance: the agent observes the market state, selects a continuous portfolio exposure $a_t \in [-1, 1]$, and receives a scalar reward reflecting realized

return net of transaction costs. A baseline formulation of this reward is:

$$r_t = a_{t-1} \cdot \frac{P_t - P_{t-1}}{P_{t-1}} - c \cdot |a_t - a_{t-1}|$$

where a_{t-1} is the exposure held during the period, P_t is price at the end of the period, and $c \cdot |a_t - a_{t-1}|$ is the transaction cost proportional to the size of the position change. The agent learns a deterministic policy $\mu_\theta : \mathcal{S} \to \mathcal{A}$ to maximize expected cumulative reward:

$$\max_\theta \mathbb{E}\left[\sum_{t=1}^{T} \gamma^{t-1} r_t\right]$$

In trading, two properties are essential for the reward to work well in practice: the signal should be risk-adjusted so the agent does not maximize returns by taking excessive volatility, and it should be computable recursively so that no look-ahead into future returns is required at each training step. The following sections explain why the baseline formulation above requires refinement in a high-frequency setting, and how the Differential Sharpe Ratio addresses both requirements.

4.3.1. Why Raw Log-Return Is Insufficient

The simplest reward is the single-step log-return:

$$r_t = \log \frac{P_t}{P_{t-1}} \cdot a_{t-1}$$

At tick frequency, this signal is dominated by microstructure noise. Individual price changes on an MBP-10 order book are a fraction of the spread. The agent cannot distinguish genuine directional movement from quote flickering in a single step.

Training on raw log-return produces highly variable gradient estimates. It also tends to reward high-turnover behavior regardless of risk.

A related alternative is the Sharpe ratio computed over a fixed rolling window. This approach requires storing the full window of past returns at each step. It also introduces a sharp discontinuity when old returns drop out of the window.[7]

[7] When the oldest return in a fixed window is unusually large or small, its removal produces an abrupt shift in the estimated mean and variance — and therefore in the Sharpe ratio — that is unrelated to the

Neither property is desirable in an online RL setting.

4.3.2. Differential Sharpe Ratio

The Differential Sharpe Ratio (Moody and Saffell 1998) resolves both issues. It is defined as the first-order approximation of how the current return R_t changes the exponentially weighted Sharpe ratio:

$$D_t = \frac{B_{t-1}\,\Delta A_t - \frac{1}{2}A_{t-1}\,\Delta B_t}{\left(B_{t-1} - A_{t-1}^2\right)^{3/2}}$$

where A_t and B_t are exponentially weighted estimates of the first and second moments of returns:

$$A_t = \eta R_t + (1-\eta)\,A_{t-1}, \qquad B_t = \eta R_t^2 + (1-\eta)\,B_{t-1}$$

and $\Delta A_t = A_t - A_{t-1}$, $\Delta B_t = B_t - B_{t-1}$. At $t = 0$, $A_0 = B_0 = 0$.

The smoothing parameter η controls the effective memory of the estimator: larger η makes the estimate more reactive to recent returns, smaller η averages over a longer history. The effective window length is approximately $1/\eta$ steps. In the empirical experiments, $\eta = 0.01$, giving an effective memory of roughly 100 ticks — long enough to smooth microstructure noise while short enough to adapt to intraday regime shifts.

The DSR has two properties that make it well-suited to this setting. First, it penalizes return volatility intrinsically. A sequence of returns with the same mean but higher variance produces a lower DSR. This discourages the agent from achieving returns through high-variance position flipping.

Second, the DSR is recursively computable from A_{t-1} and B_{t-1} alone. This means no fixed window, no look-ahead, and no discontinuity as old observations expire.

current action. The agent cannot distinguish this accounting artifact from a genuine change in performance, introducing spurious gradient variance. Exponential weighting in the DSR avoids this because old returns decay continuously rather than dropping off a cliff.

4.3.3. Reward and Financial Return Are Not the Same

A shaped reward such as DSR is used to stabilize learning. However, it is not equivalent to the financial performance metrics reported in Chapter 6.

The Sharpe ratio, annualized return, and drawdown statistics reported in the evaluation chapter are computed from the realized log-return series. These come from the deterministic policy rollout. They are not computed from the cumulative DSR accumulated during training.

Conflating the two would invalidate the empirical comparison. For example, treating a high training DSR as evidence of a strong Sharpe ratio on the test set would be incorrect.

4.4. Value Function

Chapter 3 introduced the state-value function $V^\pi(s)$ and action-value function $Q^\pi(s,a)$ as the expected cumulative discounted return under policy π. This section describes how TD3 instantiates the Q-function in practice, focusing on the design choices that distinguish it from a vanilla actor-critic.

4.4.1. Twin Critics

TD3 maintains two independent Q-networks Q_{θ_1} and Q_{θ_2}. The Bellman target is computed using the minimum of the two estimates:

$$y_t = r_t + \gamma \min_{i=1,2} Q_{\theta'_i}(s_{t+1}, \tilde{a}_{t+1})$$

where θ'_i are slowly updated target network parameters and $\tilde{a}_{t+1} = \mu_{\phi'}(s_{t+1}) + \epsilon$ is the target action with smoothing noise. Taking the minimum counteracts the overestimation bias that arises when bootstrapped targets are computed from the same network being updated — a failure mode documented in DDPG and addressed here following Fujimoto, Hoof, and Meger (2018).

Both critics are trained by minimizing the TD error against the shared target y_t. The actor is updated using only the first critic's gradient:

$$\nabla_\phi J(\phi) = \mathbb{E}_{s\sim\mathcal{D}}\left[\nabla_a Q_{\theta_1}(s,a)\big|_{a=\mu_\phi(s)} \nabla_\phi \mu_\phi(s)\right]$$

The second critic contributes only through the conservative Bellman target, not directly to the policy gradient.

4.4.2. Loss Function

The critics are trained with Smooth L1 (Huber) loss rather than MSE:

$$\mathcal{L}(\delta) = \begin{cases} 0.5\,\delta^2 & |\delta| \leq 1 \\ |\delta| - 0.5 & \text{otherwise} \end{cases}$$

where $\delta = y_t - Q_\theta(s_t, a_t)$ is the TD error. The linear tail limits the influence of large TD errors that occur during volatile market episodes, providing robustness that MSE does not.

4.5. Policy

4.6. Discount Factor

The discount factor $\gamma = 0.99$ is used throughout this thesis. The role of γ in the value function is defined in Chapter 3; this section justifies the specific value chosen.

With $\gamma = 0.99$ the effective planning horizon is approximately $1/(1-\gamma) = 100$ steps. On the HFT data used here — where each step corresponds to a short intraday interval — this horizon covers a meaningful window of market activity without extending so far that value estimates accumulate excessive variance from stochastic rewards.

$\gamma = 0.99$ is held constant across all experiments to isolate the effect of other design choices (state representation, reward function). Sensitivity to γ is a natural direction for future work, particularly whether lower values (0.9–0.95) improve performance in choppy, mean-reverting regimes.

4.7. Optimization Hyperparameters

4.7.1. Learning Rates

Both actor and critic use Adam (Kingma and Ba 2014) with a learning rate of $\alpha = 0.0001$. This is deliberately conservative relative to the 3×10^{-4} default common in continuous-control benchmarks. Preliminary experiments with more aggressive rates produced Q-value explosions and policy collapse — failure modes consistent with the high noise-to-signal ratio

of tick-frequency rewards and the sensitivity of bootstrapped TD targets to large gradient steps. Stability was prioritized over sample efficiency.

Actor and critic use the same nominal rate, but TD3's policy delay (every $d = 2$ critic updates) effectively halves the actor's update frequency, providing additional stability without a separate hyperparameter.

4.7.2. Weight Decay

L2 weight decay of $\lambda = 2 \times 10^{-6}$ is applied to both networks. The magnitude is intentionally small — sufficient to discourage memorization of training-period noise without restricting the network's capacity to represent nonlinear market dynamics. No further regularization (dropout, batch normalization) is used, keeping the architecture parsimonious.

4.7.3. Scope

The reported learning rate and weight decay define the experimental configuration used throughout the experiments. They were chosen to maintain stable critic targets and avoid policy collapse in preliminary runs, which makes them appropriate for the present high-frequency training setup. A broader hyperparameter study could refine these values further, but that analysis lies beyond the scope of the current empirical comparison.

5. Implementation of the Trading Agent

This chapter describes how the design choices from Chapter 4 are realized in the experimental pipeline. It covers the simulation architecture, data preparation, code organization, and the TD3 training workflow.

5.1. Simulation Architecture

The empirical workflow is organized through three functional layers that together implement a single-asset trading simulation.

Market data and feature construction. The agent observes states derived from high-frequency AAPL order-book data. Raw order-book information is transformed into a vector of microstructure features. This layer is responsible for loading and validating the market dataset, preserving chronological ordering, constructing derived features, separating learning features from the price series used for portfolio valuation, and creating train, validation, and test splits without leakage. It is important because reinforcement learning in financial settings is highly sensitive to timestamp errors and inconsistencies between engineered features and the price series used by the simulation.

Decision layer. The RL agent maps the current state representation to a bounded continuous action interpreted as target portfolio exposure. The agent learns action selection, turnover, and reward trade-offs jointly through environmental feedback — there is no separate forecasting step or labeled-action prior. The quality of the decision is evaluated through the reward process generated by the environment.

Execution and portfolio accounting. The agent's chosen action is translated into a portfolio update under explicit assumptions about pricing, transaction costs, and liquidity. This layer abstracts the operational complexity of a production trading stack — order management, venue selection, smart order routing, fill handling — into a controlled backtesting mechanism. If the agent performs well in this simulated setting, the result supports a claim about policy learning under the specified assumptions, not a claim that the strategy is immediately deployable without further execution modeling.

5.2. Simulation Assumptions

The following assumptions govern the trading simulation throughout all experiments.

Zero market impact and no order-book interaction. Agent trades are assumed not to move quoted prices, consume displayed depth, or alter subsequent order-book states. The environment therefore treats the agent as price-taking rather than market-shaping. This is defensible only while position sizes remain small relative to displayed order-book depth, which is the case for the AAPL MBP-10 dataset used here.

Mid-price execution. Trades are valued at the mid-price rather than at executable bid/ask quotes. This avoids explicit spread-crossing and simplifies reward accounting, but it implies that reported backtest returns are an upper bound on achievable net performance. Real execution must cross the spread, and realized returns are further reduced by maker/taker fees, queue position, and partial fills.

Zero latency. Observations are assumed to be acted on immediately, with no transport, decision, or matching-engine delay between state observation and execution. This is appropriate for a study focused on strategy learning rather than low-latency execution engineering. In production HFT systems, such delays can invalidate short-lived signals.

Bounded continuous exposure. Position constraints are enforced via the continuous action interval $[-1, 1]$, preventing unbounded leverage.

Transaction fee. A configurable fee parameter is applied per position change. Baseline runs use a no-fee setting; sensitivity experiments vary this parameter.

Narrow data scope. Training, validation, and testing are conducted on a single three-day window of AAPL data (February 25–27, 2026). This scope means the agent has only encountered one volatility regime, one spread environment, and one pattern of intraday seasonality. Results should be interpreted as a proof of concept rather than as evidence of generalizability across market conditions or time periods.

These assumptions collectively make the simulation tractable and the results interpretable as learning-dynamics benchmarks rather than deployment-ready performance estimates. The following components of a production trading system are explicitly outside the scope of this study: smart order routing across venues, partial fills and queue-position effects, execution algorithms such as VWAP or TWAP, detailed market-impact modeling, and production monitoring or brokerage integration. These omissions define the gap between the controlled experimental framework and deployment-ready trading-system performance.

5.3. Data Preparation

5.3.1. Asset Selection

The experiments use equity data from four Nasdaq-listed blue-chip instruments: AAPL (Apple Inc.), MSFT (Microsoft Corp.), TSLA (Tesla Inc.), and NVDA (NVIDIA Corp.). All four are among the most liquid equities in US markets, with tight spreads, deep displayed depth, and continuous two-sided quoting throughout the trading session. This liquidity profile ensures that the price-taking assumption embedded in the simulation — that agent trades do not move quoted prices — is empirically defensible, and that the microstructure features derived from the order book reflect genuine supply and demand rather than thin-market artifacts.

The choice to train a single pooled model across multiple instruments, rather than a separate model per instrument, is a deliberate design decision. Securities within the same asset class that trade on the same exchange, under the same tick-size regime, and with comparable order-flow dynamics share a common microstructure regime. The residual cross-sectional variation — differences in volatility level, beta, or sector exposure — is dominated by this shared structure. Within such a homogeneous class, LOB feature distributions are comparable after standard normalization, making it feasible for a single policy network to learn market-wide order-book dynamics rather than instrument-specific quirks. Pooling across N securities multiplies the effective training sample by a factor of N, reducing the risk of overfitting to idiosyncratic patterns in any single stock's order-flow sequence. It also encourages the model to capture universal microstructure signals — spread dynamics, order-flow imbalance, and depth asymmetries — that transfer across equities within the same class. The feature pipeline supports this directly: all inputs are z-score normalized per security, and the microstructure features in the state space — order-flow imbalance, relative spread, depth slope, queue imbalance — are inherently relative measures that carry the same semantic meaning regardless of the underlying price level or absolute volume.

A further consideration is access symmetry. Because all four instruments trade on a centralized, regulated exchange with public pre-trade transparency, all market participants observe the same order book. This contrasts with OTC markets or less liquid instruments where information asymmetry and differential access could confound the learned signal. The goal is to isolate the contribution of the learning algorithm and feature engineering, not to exploit

a structural information advantage.

The data were obtained from Nasdaq via the DataBento API, using the MBP-10 (Market By Price, 10 levels) feed — a standardised L3 order book format that provides full depth-of-book snapshots at the event level, including resting quantities and order counts at each price level. This granularity is necessary for constructing the microstructure features described in Chapter 4 and listed in Appendix A.

5.3.2. Input Format

The pipeline consumes time-indexed LOB snapshot data. Each row represents an order-book event with multi-level bid/ask prices, sizes, and order counts, plus trade descriptors used for order-flow features. The core column groups are summarized below.

Table 2: Input schema used in the empirical pipeline

Column group	Description	Role
`timestamp`	Chronological event index	Ordering, temporal features
`bid_px_NN`, `ask_px_NN`	Bid/ask prices at level `NN`	Spread, mid-price, slope, valuation
`bid_sz_NN`, `ask_sz_NN`	Resting size at level `NN`	Imbalance, depth, OFI, queue features
`bid_ct_NN`, `ask_ct_NN`	Order count at level `NN`	Order count imbalance
`action`, `side`, `size`	Event type, aggressor side, size	Signed trade flow features
`close` (auxiliary)	Mid-price-like pricing series	Reward and portfolio valuation

5.3.3. Preprocessing and Splitting

Raw data is validated for chronological order, required column presence, non-negative prices, and non-crossed best quotes before any feature engineering. This stage is conservative: its purpose is correctness, not statistical filtering.

The dataset is then split chronologically into training, validation, and test subsets. Shuffling is not used because it would destroy temporal structure and introduce information from future

periods into earlier training samples. The training split is used to fit model parameters; the validation split supports configuration comparison and development diagnostics; the test split is reserved for final out-of-sample reporting and does not influence any model selection decisions.

The main experiment uses a training split of 15,000 order-book events, with approximately 2,500 events each in the validation and test splits — 20,000 observations in total, spanning three consecutive trading sessions from February 25 to February 27, 2026, filtered to US regular market hours. The narrow three-day window is a recognized limitation of the current empirical design: the agent's learned behavior has not been exposed to different volatility regimes, spread environments, or intraday seasonality patterns beyond this specific period, and the generalizability of the results is bounded accordingly.

5.3.4. Feature Normalization and Leakage Prevention

Normalization statistics (mean and standard deviation for z-score scaling) are estimated on the training split only and applied unchanged to the validation and test splits. Fitting normalization on the full dataset would allow future volatility regimes to influence the training representation, inflating apparent performance. Cyclical time encodings (hour-of-day sine and cosine) are already bounded in $[-1, 1]$ and are not normalized. Raw price columns used for portfolio accounting are kept in their original units and are never replaced by normalized values.

5.3.5. Feature and Price Column Separation

A strict separation is maintained between the observation features passed to the learning algorithm and the price inputs used by the portfolio simulation. The learning features are normalized and engineered; the pricing columns are economically interpretable and used only for reward and valuation computation. Mixing these two roles can produce invalid reward calculations or cause normalized prices to appear in financial metrics.

In the current implementation, the auxiliary pricing series used for portfolio valuation plays the role of a mid-price proxy rather than an executable bid/ask quote. This simplifies reward accounting but makes the resulting backtest performance optimistic relative to realistic spread-crossing execution.

5.4. Implementation and Training Pipeline

5.4.1. Configuration-Driven Design

Each experiment is fully specified by an external YAML scenario file covering data paths, feature subsets, reward type, environment settings, and optimization hyperparameters. No source-code changes are required to switch between experimental variants. A pre-training validation stage checks path availability, feature definitions, split feasibility, and column compatibility before any computation begins, preventing wasted runs from misconfigured experiments. A condensed example configuration is provided in Appendix C.

5.4.2. Feature Pipeline

Feature engineering is registry-driven: feature names in the scenario file resolve to computation functions, allowing scenario-level composition without editing training code. This design is central to systematic feature subset comparisons. Normalization parameters are estimated on the training split only and applied without re-fitting to the validation and test data, preserving causal integrity across the chronological split.

5.4.3. Training Loop

The processed training split is used to construct the trading environment with a continuous action space $a_t \in [-1, 1]$ representing target portfolio exposure. Training follows the TD3 procedure described in Chapter 3: experience is collected into a replay buffer, twin critics are updated from sampled mini-batches, and the actor is updated every d critic steps via the deterministic policy gradient. Target networks track the main networks through soft updates with coefficient τ. Exploration noise is applied during training and removed at evaluation time.

The notation follows the conventions established in Chapter 3. μ_θ is the actor (deterministic policy) with parameters θ; Q_{ϕ_i} are the two critic networks with parameters ϕ_i; primed symbols (θ', ϕ_i') denote the corresponding slowly updated target networks. $\mathcal{D}$ is the replay buffer storing past transitions (s, a, r, s'). $\varepsilon \sim \mathcal{N}(0, \sigma_{\text{explore}})$ is the exploration noise added to actions during training only; σ_{noise} and c are the target policy smoothing standard deviation and clip bound respectively. γ is the discount factor, τ the soft-update coefficient, α the actor learning rate, and d the policy delay (critic updates per actor update). Concrete values for

Algorithm 1 TD3 Trading Experiment Workflow

Require: Experiment configuration (data path, feature set, hyperparameters)
Ensure: Trained policy μ_θ, evaluation metrics

Load and validate configuration; check paths and split feasibility
Load L3 order book dataset; sort chronologically
Split data into train / validation / test subsets (chronological)
Fit feature pipeline on training split only
Transform all splits using fitted pipeline
Build training environment from processed training data
Initialise actor μ_θ, twin critics Q_{ϕ_1}, Q_{ϕ_2}, target networks $\mu_{\theta'}$, $Q_{\phi_1'}$, $Q_{\phi_2'}$, replay buffer $\mathcal{D}$
for each step $t = 1, \dots, \texttt{max_steps}$ **do**
    $a_t \leftarrow \mu_\theta(s_t) + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0,\, \sigma_{\text{explore}})$
    Execute a_t; observe r_t, s_{t+1}; store (s_t, a_t, r_t, s_{t+1}) in $\mathcal{D}$
    Sample mini-batch $\mathcal{B} \sim \mathcal{D}$
    Compute smoothed target action: $\tilde{a}' \leftarrow \mu_{\theta'}(s') + \text{clip}(\mathcal{N}(0, \sigma_{\text{noise}}),\, \pm c)$
    Compute Bellman target: $y \leftarrow r + \gamma \cdot \min_{i=1,2} Q_{\phi_i'}(s',\, \tilde{a}')$
    Update both critics by minimising $\mathcal{L}_{\text{Smooth-L1}}\big(y - Q_{\phi_i}(s, a)\big)$
    if $t \bmod \texttt{policy_delay} = 0$ **then**
        Update actor: $\theta \leftarrow \theta + \alpha \nabla_\theta Q_{\phi_1}(s,\, \mu_\theta(s))$
        Soft-update targets: $\phi_i' \leftarrow \tau\phi_i + (1-\tau)\phi_i'; \quad \theta' \leftarrow \tau\theta + (1-\tau)\theta'$
    end if
    Checkpoint and log metrics at configured intervals
end for
Evaluate deterministic policy μ_θ on test split
Compute financial performance metrics from realised return series

all parameters are listed in Appendix B.

5.4.4. Evaluation

Final performance is reported on the held-out test split using a dedicated evaluation environment. Two output streams are kept strictly separate. First is the training reward, which serves as an optimization diagnostic. Second is the realized portfolio return series from which financial metrics are computed.

Conflating these would invalidate the empirical comparison. For example, treating cumulative DSR as a Sharpe ratio estimate would be incorrect. Metrics derived from the test rollout are described in Chapter 6.

6. Results [PLACEHOLDER]

This chapter reports the evidence for the thesis-facing experiment: the TD3 agent trained on high-frequency AAPL order-book-derived features under the state-space design defined in Chapters 4 and 5. It is organized in three parts: statistical validation of repeated-run variability, robustness assessment under nearby design changes, and final performance evaluation on the held-out split.

6.1. Benchmark Strategies [PLACEHOLDER]

Performance is assessed relative to five benchmark strategies. These benchmarks span passive exposure, execution-style references, and a stochastic baseline, together providing a comparison surface for the directional TD3 agent.

Buy-and-hold maintains a constant fully long position ($a_t = 1.0$) throughout the evaluation period, capturing the full market return of the asset. It is the primary economic benchmark: any active strategy that fails to outperform buy-and-hold on a risk-adjusted basis has not demonstrated useful directional skill beyond passive exposure.

Short-and-hold maintains a constant fully short position ($a_t = -1.0$). Its inclusion is symmetric: if the evaluation window exhibits negative drift, short-and-hold is the passive strategy that benefits, and the agent should demonstrate that its advantage does not stem merely from correctly calling the direction of a persistent trend.

TWAP (Time-Weighted Average Price) distributes position changes uniformly across time. **VWAP (Volume-Weighted Average Price)** distributes them in proportion to historical volume patterns. Both are standard execution benchmarks used by institutional traders to minimize market impact on predetermined order schedules. It is important to note that TWAP and VWAP are execution algorithms rather than directional strategies — they describe how to work an order, not whether to be long or short. Their inclusion reflects standard practice in trading-system evaluation; however, outperforming them has a different interpretation than outperforming buy-and-hold, and they serve as lower-bar references rather than directional competitors.

Random selects actions uniformly from $[-1, 1]$ at each step without reference to market state. Evaluation uses 100 independent random-seed trials and reports aggregate performance to obtain a stable stochastic baseline. Failure to consistently outperform a random policy would

indicate that the agent has not learned a meaningful relationship between observations and actions.

6.2. Evaluation Protocol [PLACEHOLDER]

To keep the rendered thesis aligned with the recorded experiments, tables and figures are loaded from exported result snapshots generated by the experiment tracking workflow. When no exported snapshot is available, the chapter falls back to the local tracking store.

Two result views are used throughout:

- a monitoring view of the latest active run, when training is still in progress
- an evaluation view of the latest finished run with test-set artifacts

6.3. Experiment Snapshot [PLACEHOLDER]

```
No runs found for experiment: TRADINGENV_TD3_AAPL_HFT_STATE_SPACE

No active run detected for the selected experiment.

No evaluation comparison plot artifact available yet.
```

6.4. Statistical Validation [PLACEHOLDER]

This section focuses on uncertainty arising from the training process itself. For the present AAPL high-frequency TD3 setup, repeated runs can diverge because of random initialization, replay sampling, and exploration noise even when the dataset and configuration are fixed. The purpose of statistical validation here is therefore narrower than overall evaluation: it quantifies run-to-run dispersion and tests whether observed benchmark differences are consistent rather than accidental.

The analysis operates on exported thesis result snapshots so that the rendered chapter stays tied to recorded experiment artifacts.

6.4.1. Statistical Test Design [PLACEHOLDER]

For each benchmark strategy defined in the preceding section, the evaluation pipeline runs five statistical tests on the held-out return series to assess whether the observed performance difference is consistent with a null hypothesis of equal means or equal distributions.

Student's t-test tests whether the mean step-level return of the agent's policy differs significantly from the benchmark mean. The test assumes approximate normality and independence of observations; at tick frequency both assumptions are at best approximations, so its result is treated as indicative rather than conclusive.

Sharpe ratio bootstrap and **Sortino ratio bootstrap** construct confidence intervals on the difference in risk-adjusted return between the agent and each benchmark using 10,000 bootstrap resamplings of the return series. Bootstrap methods make no distributional assumption on returns, making them more appropriate than parametric alternatives for high-frequency financial data.

Mann-Whitney U test is a non-parametric rank test that evaluates whether the agent's return distribution stochastically dominates the benchmark distribution, without assuming normality or equal variances.

Permutation test randomly permutes the agent's return series 10,000 times to build an empirical null distribution and assess whether the observed test statistic is extreme relative to what would occur by chance under the null of no systematic relationship between observations and returns.

All tests use a 95% confidence level. The random baseline is evaluated separately over 100 independent trials with different random seeds, and the aggregate distribution of these trials forms the null for that comparison. Test results are logged as MLflow artifacts and are most informative when considered jointly: no single test is treated as definitive, but consistent evidence across multiple tests — especially the non-parametric ones — provides stronger support for the performance claim.

6.4.2. Purpose and Scope of Statistical Validation [PLACEHOLDER]

In the current work, statistical validation serves three narrower purposes. First, it quantifies dispersion across repeated runs of the same AAPL HFT configuration under different random seeds. Second, it supports comparison between alternative state representations or reward choices when those comparisons are available. Third, it prevents overinterpretation of a single run whose performance may reflect favorable stochastic training dynamics rather than repeatable policy quality.

The present thesis distinguishes between two classes of quantities:

1. training-time monitoring metrics, which are useful for diagnosing learning dynamics but are not sufficient as final evidence of financial performance;
2. final evaluation metrics, which summarize realized portfolio behavior and are the primary basis for empirical conclusions.

This distinction is essential because low optimization loss or improving training diagnostics do not necessarily imply superior out-of-sample trading performance.

6.4.3. Statistical Significance Summary [PLACEHOLDER]

The following table summarizes the results of five statistical tests comparing the TD3 agent against each benchmark strategy. For each test × benchmark combination, the p-value is reported along with a significance indicator at the 95% confidence level ($p < 0.05$). The tests span parametric and non-parametric approaches, providing complementary evidence on the performance difference.

`Placeholder values shown - run full experiment pipeline to populate.`

	Test	Benchmark	p-value	Significant ($p < 0.05$)
0	Student's t-test	Buy & Hold	0.0008	Yes
1	Student's t-test	Short & Hold	0.0231	Yes
2	Student's t-test	TWAP	0.0087	Yes
3	Student's t-test	VWAP	0.0324	Yes
4	Student's t-test	Random	0.0001	Yes
5	Sharpe Bootstrap	Buy & Hold	0.0012	Yes
6	Sharpe Bootstrap	Short & Hold	0.0285	Yes
7	Sharpe Bootstrap	TWAP	0.0103	Yes
8	Sharpe Bootstrap	VWAP	0.0389	Yes
9	Sharpe Bootstrap	Random	0.0002	Yes
10	Sortino Bootstrap	Buy & Hold	0.0009	Yes
11	Sortino Bootstrap	Short & Hold	0.0256	Yes
12	Sortino Bootstrap	TWAP	0.0095	Yes
13	Sortino Bootstrap	VWAP	0.0351	Yes
14	Sortino Bootstrap	Random	0.0001	Yes
15	Mann-Whitney U	Buy & Hold	0.0015	Yes

	Test	Benchmark	p-value	Significant ($p < 0.05$)
16	Mann-Whitney U	Short & Hold	0.0312	Yes
17	Mann-Whitney U	TWAP	0.0124	Yes
18	Mann-Whitney U	VWAP	0.0423	Yes
19	Mann-Whitney U	Random	0.0003	Yes
20	Permutation Test	Buy & Hold	0.0011	Yes
21	Permutation Test	Short & Hold	0.0278	Yes
22	Permutation Test	TWAP	0.0099	Yes
23	Permutation Test	VWAP	0.0367	Yes
24	Permutation Test	Random	0.0002	Yes

6.4.4. Run-Level Sample Summary [PLACEHOLDER]

The following table summarizes currently available finished runs for the selected experiment. These values should be interpreted as a sample of outcomes under the configured workflow rather than as definitive performance estimates.

```
No finished runs available for experiment: TRADINGENV_TD3_AAPL_HFT_STATE_SPACE
```

6.4.5. Descriptive Statistics and Uncertainty Estimates [PLACEHOLDER]

For repeated runs of the same configuration, descriptive statistics provide the first indication of stability. Here, quantities such as the sample mean, standard deviation, and range are summaries of variability under the reported protocol, not as universal claims about future market performance. Repeated backtests quantify sensitivity to training randomness more than they resolve uncertainty about the underlying market process.

```
No completed runs available for statistical summaries.
```

6.4.6. Comparative Validation Strategy for Thesis-Grade Results [PLACEHOLDER]

When comparative experiments are reported, they should be interpreted under a matched design: the data split, cost assumptions, and evaluation procedure must remain fixed while only the feature set, reward specification, or other target variation is changed. Within that design, the preferred interpretation hierarchy is:

1. effect size (difference in mean or median performance),
2. dispersion (stability across runs),
3. confidence interval overlap or formal hypothesis testing,
4. economic plausibility (e.g., turnover, drawdown, net performance after costs).

This ordering matters because statistically detectable differences can still be economically unimportant if they coincide with unstable turnover, weak net returns after costs, or poor drawdown behavior.

6.4.7. Statistical Risks and Interpretation Caveats [PLACEHOLDER]

Several risks remain when interpreting these summaries. Repeated runs on the same historical period are not independent samples from a stable data-generating process; they quantify training stochasticity more than market uncertainty. In addition, repeated tuning on the same validation period can bias later comparisons, and reward-based metrics can diverge from financial objectives when the reward is a shaped training signal rather than realized return.

Accordingly, statistical validation is treated here as a method for ranking the credibility of observed results under a controlled protocol, not as proof of persistent real-world profitability. Final claims must therefore be read together with robustness assessment and economic interpretation.

6.5. Robustness Assessment [PLACEHOLDER]

This section shifts from training stochasticity to sensitivity with respect to modeling choices. The question is whether the conclusions for the current AAPL high-frequency TD3 setup would change materially under nearby alternatives in feature specification, reward definition, transaction-cost assumptions, or split design.

The goal is to separate results that appear structural from results that may depend too heavily on one narrow experimental configuration.

6.5.1. Why Robustness Matters in This Thesis [PLACEHOLDER]

In this work, robustness matters because the agent is trained on a noisy microstructure-driven reward stream and evaluated in a simplified execution environment. Apparent improvements

may therefore come from favorable training dynamics or feature-set idiosyncrasies rather than from a genuinely more useful policy. A result is treated as robust only if it remains directionally consistent under reasonable changes in seed, state representation, reward specification, and cost assumptions, while preserving interpretable turnover and drawdown behavior.

6.5.2. Current Evaluation Context [PLACEHOLDER]

The following snapshot provides the current experiment context used in the automated Chapter 6 result wiring. It is included to make the robustness discussion explicit about which configuration the displayed results refer to.

```
No finished runs available for experiment: TRADINGENV_TD3_AAPL_HFT_STATE_SPACE
```

6.5.3. Robustness Dimensions Considered in This Thesis [PLACEHOLDER]

The robustness framework is centered on five dimensions most directly connected to the current empirical design:

1. **Random-seed sensitivity**: whether performance is stable across repeated runs of the same configuration.
2. **Feature-set sensitivity**: if the richer state-space design improves results relative to simpler snapshot-heavy alternatives without introducing unstable behavior.
3. **Reward-function sensitivity**: whether conclusions depend on the specific training objective used by the agent.
4. **Transaction-cost sensitivity**: whether apparent gains survive realistic fee assumptions rather than being generated by excessive turnover.
5. **Data-split sensitivity**: whether results remain similar under reasonable changes in the chronological split or training-window length.

These dimensions are sufficient to test if the conclusions here are specific to one narrow configuration or remain credible under nearby variations.

6.5.4. Robustness Matrix for Experimental Reporting [PLACEHOLDER]

The table below defines a practical robustness matrix for the evaluation part of the thesis. It can be used as a reporting checklist when final results are compiled.

	Dimension	Control Variable	Primary Risk Addressed
0	Random seed	seed	Stochastic training path sensitivity
1	Feature subset	env.feature_columns / feature pipeline	State representation redundancy and overfitt
2	Reward function	reward_type	Objective-function dependence
3	Transaction costs	trading_fees	Turnover-driven performance inflation
4	Train split length	train_size / split design	Sample size and regime exposure sensitivity

6.5.5. Interpreting Noisy Training Curves in Robustness Terms [PLACEHOLDER]

Noisy training curves are common in actor-critic methods and should not be treated as the primary robustness criterion. In the present workflow, the more informative question is whether the same configuration produces similar evaluation outcomes across repeated runs and whether qualitative behavior, such as persistent overtrading or directional bias, changes systematically when the feature set, reward, or cost assumptions are varied.

This matters in the current work because one of the central comparisons concerns simpler versus broader order-book-derived state representations. If the richer feature set improves returns but materially increases instability or turnover, the result is weaker than a modest but more stable improvement.

6.5.6. Methodological Limits of the Current Robustness Snapshot [PLACEHOLDER]

The current snapshot-based Chapter 6 integration supports reproducible loading of run summaries and evaluation artifacts, but robustness conclusions remain conditional on the set of experiments already executed and exported. A single finished run can illustrate workflow behavior, but it cannot support strong claims about robustness across seeds, feature sets, or cost regimes.

For this reason, robustness findings in the final thesis should be framed only to the extent supported by repeated runs and matched comparison settings. The framework in this section provides the structure for that interpretation and helps prevent post-hoc emphasis on isolated favorable outcomes.

6.6. Performance Evaluation [PLACEHOLDER]

This section reports the held-out performance of the latest completed run of the thesis experiment. The aim is not just to list metrics, but to judge whether the TD3 agent's behavior remains economically coherent once returns, turnover, and path-dependent risk measures are inspected together. Results are loaded from exported thesis snapshots, with a local tracking-store fallback, so that the rendered chapter stays tied to recorded evaluation artifacts.

6.6.1. Selected Metrics from the Latest Completed Run [PLACEHOLDER]

`Placeholder values shown - run full experiment pipeline to populate.`

	Metric	Value
0	Total Return	0.28%
1	CAGR	22.36%
2	Annualized Volatility	12.34%
3	Sharpe Ratio	1.8234
4	Sortino Ratio	2.4123
5	Max Drawdown	-0.16%
6	Win Rate	56.21%
7	Profit Factor	1.3845
8	VaR (95%)	-0.0021
9	CVaR (95%)	-0.0034

6.6.2. Benchmark Comparison [PLACEHOLDER]

The TD3 agent is evaluated against five benchmark strategies spanning passive exposure, execution-style references, and a stochastic baseline. The table below summarizes key risk-adjusted metrics for each strategy, with the agent's performance highlighted in the first row.

`Placeholder values shown - run full experiment pipeline to populate.`

	Strategy	Total Return	Annualized Return (CAGR)	Annualized Volatility	Sharpe Ratio	S
0	TD3 Agent	0.28%	22.36%	12.34%	1.8234	2.
1	Buy & Hold	-0.12%	-9.80%	14.40%	-0.6800	-0

	Strategy	Total Return	Annualized Return (CAGR)	Annualized Volatility	Sharpe Ratio	Sorti
2	Short & Hold	0.12%	9.80%	14.40%	0.6800	0.910
3	TWAP	-0.08%	-6.50%	14.80%	-0.4400	-0.52
4	VWAP	-0.07%	-5.70%	15.00%	-0.3800	-0.45
5	Random	0.01%	0.80%	16.80%	0.0500	0.060

6.6.3. Evaluation Metadata Snapshot [PLACEHOLDER]

6.6.4. Evaluation Plots [PLACEHOLDER]

```
No finished run available to display evaluation plots.
```

6.6.5. Discussion: Reading Performance in the Context of State Design [PLACEHOLDER]

The key interpretive issue is not whether the agent attains a single high metric, but whether the reported behavior is consistent with the order-book-informed state design proposed in Chapter 4. Because the current work studies order-book-derived state representations, performance should be read jointly with turnover, drawdown, and action behavior rather than through return metrics alone.

This matters especially when comparing simpler snapshot-heavy feature sets with broader state representations that also encode flow, timing, and spread-regime information. A richer state is only useful if it improves held-out behavior in an economically credible way. If additional features increase turnover, produce fragile policy behavior, or improve reward diagnostics without improving net returns, they do not strengthen the empirical case for the proposed state design.

For that reason, the performance tables in this section reflect the quality of the overall decision pipeline: state representation, reward specification, and policy learning together. The central question is whether the broader microstructure-aware feature design yields more stable and more interpretable held-out behavior than simpler alternatives, not whether any single metric is maximized in isolation.

7. Conclusions and Future Work [PLACEHOLDER]

This thesis examined if a continuous-control deep RL agent using order-book-derived microstructure features can achieve economically meaningful held-out performance in a high-frequency single-asset setting. Within the specified experimental design, the answer is affirmative. The TD3 agent outperformed the evaluated benchmark strategies on risk-adjusted return metrics, and the richer microstructure-aware state representation produced a clear improvement over the simpler snapshot-based baseline. These findings support the central claim of the thesis: the informational content of the limit order book can be translated into state features that materially improve the behavior of an RL trading agent.

The empirical contribution is accompanied by a methodological one. The thesis developed an end-to-end and reproducible pipeline covering feature engineering from 10-level order book data, configuration-driven experiment management, TD3 training with a Differential Sharpe Ratio reward, and a strict separation between training diagnostics and held-out financial evaluation. This methodological contribution matters alongside the empirical one: it provides a reproducible framework for studying RL on limit order book data under explicit simulation assumptions and chronological evaluation discipline.

At the same time, the conclusions must be interpreted within the limits of the experimental setup. The simulation uses a mid-price execution proxy rather than executable bid/ask fills, assumes zero market impact and zero latency, and is evaluated on a narrow three-day AAPL window with a single random seed. These choices were deliberate because they kept the experimental design tractable and made it possible to determine if the learning problem itself is feasible. They also place a clear boundary on the claim being made. The thesis demonstrates feasibility, not deployability. The observed edge cannot yet be treated as evidence of a production-ready trading strategy, because realistic execution frictions may materially reduce or eliminate it.

These limitations point directly to the most important implications for future research and for practical system design. First, transaction-cost sensitivity must be treated as a binding constraint rather than as a minor robustness check: any apparent edge that disappears once spread-crossing costs, fees, queue uncertainty, or latency are introduced should not be interpreted as economically meaningful. Second, feature engineering deserves priority over marginal changes in model architecture, because the results suggest that policy quality is

driven primarily by the predictive content of the state representation. Third, stronger claims about generalization require broader temporal and cross-market evaluation, including longer windows, walk-forward validation, multiple random seeds, and eventually cross-asset testing.

The most direct directions for future work therefore follow from these constraints. The same agents should be re-evaluated under explicit bid/ask execution rules, calibrated fee schedules, and slippage or market-impact models. The evaluation scope should be extended to longer and more diverse market periods, including different volatility and liquidity regimes, with multi-seed reporting to separate genuine policy quality from favorable initialization. Natural model extensions include richer agent-state variables, broader cross-asset information, and comparisons with alternative actor-critic or sequence-model architectures. Taken together, these extensions would test whether the feasibility result established here survives under more realistic conditions.

In sum, the thesis shows that combining limit order book data, LOB-informed feature engineering, and continuous-action actor-critic learning is a productive research direction for high-frequency trading. The defensible conclusion is a narrow one: under the conditions studied here, the learning problem is tractable and the chosen representation carries enough signal to produce a measurably superior policy in a controlled setting. That result provides a foundation for subsequent work aimed at testing whether the same approach survives under more realistic assumptions.

List of Figures

Bibliography

Bellman, Richard. 1957. *Dynamic Programming*. Princeton, NJ: Princeton University Press.

Biais, Bruno, Pierre Hillion, and Chester Spatt. 1995. "An Empirical Analysis of the Limit Order Book and the Order Flow in the Paris Bourse." *Journal of Finance* 50 (5): 1655–89. https://doi.org/10.1111/j.1540-6261.1995.tb05192.x.

Boehmer, Ekkehart, Charles M Jones, Xiaoyan Zhang, and Xinran Zhang. 2021. "Tracking Retail Investor Activity." *Journal of Finance* 76 (5): 2249–2305. https://doi.org/10.1111/jofi.13033.

Bouchaud, Jean-Philippe, Marc Mézard, and Marc Potters. 2002. "Statistical Properties of Stock Order Books: Empirical Results and Models." *Quantitative Finance* 2 (4): 251–56. https://doi.org/10.1088/1469-7688/2/4/301.

Cao, Charles, Zhiwu Chen, and John M Griffin. 2004. "Informational Content of Option Volume Prior to Takeovers." *Journal of Business* 77 (6): 1073–1109. https://doi.org/10.1086/422629.

Cong, Fei. 2020. "Reward Engineering in Reinforcement Learning for Trading (Provisional Thesis Citation)."

Cont, Rama, Arseniy Kukanov, and Sasha Stoikov. 2014. "The Price Impact of Order Book Events." *Journal of Financial Econometrics* 12 (1): 47–88. https://doi.org/10.1093/jjfinec/nbt002.

Dempster, Michael A. H., and Yadolah Romahi. 2002. "Trend Following and Mean Reversion in the FX Market Using Reinforcement Learning." *Quantitative Finance* 2 (3): 159–76.

Deng, Yue, Feng Bao, Youyong Kong, Zhiquan Ren, and Qionghai Dai. 2017. "Deep Direct Reinforcement Learning for Financial Signal Representation and Trading." *IEEE Transactions on Neural Networks and Learning Systems* 28 (3): 653–64. https://doi.org/10.1109/TNNLS.2016.2522401.

Easley, David, Marcos M López de Prado, and Maureen O'Hara. 2012. "Flow Toxicity and Liquidity in a High-Frequency World." *Review of Financial Studies* 25 (5): 1457–93. https://doi.org/10.1093/rfs/hhs053.

Engle, Robert F. 2000. "The Econometrics of Ultra-High-Frequency Data." *Econometrica* 68 (1): 1–22. https://doi.org/10.1111/1468-0262.00091.

Fujimoto, Scott, Herke van Hoof, and David Meger. 2018. "Addressing Function Approximation Error in Actor-Critic Methods." *ICML*, 1587–96.

Glosten, Lawrence R. 1994. "Is the Electronic Open Limit Order Book Inevitable?" *Journal of Finance* 49 (4): 1127–61. https://doi.org/10.1111/j.1540-6261.1994.tb02450.x.

Glosten, Lawrence R, and Paul R Milgrom. 1985. "Bid, Ask and Transaction Prices in a Specialist Market with Heterogeneously Informed Traders." *Journal of Financial Economics* 14 (1): 71–100. https://doi.org/10.1016/0304-405X(85)90044-3.

Gold, Christian. 2003. "Policy Search Methods for Financial Trading (Provisional Thesis Citation)."

Haarnoja, Tuomas, Aurick Zhou, Pieter Abbeel, and Sergey Levine. 2018. "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor." *ICML*, 1861–70.

Harris, Lawrence. 1986. "A Transaction Data Study of Weekly and Intradaily Patterns in Stock Returns." *Journal of Financial Economics* 16 (1): 99–117. https://doi.org/10.1016/0304-405X(86)90044-9.

———. 1991. "Stock Price Clustering and Discreteness." *Review of Financial Studies* 4 (3): 389–415. https://doi.org/10.1093/rfs/4.3.389.

Hasselt, Hado van, Arthur Guez, and David Silver. 2016. "Deep Reinforcement Learning with Double Q-Learning." In *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 30. 1. https://doi.org/10.1609/aaai.v30i1.10295.

Jiang, Zhengyao, Dixing Xu, and Jinjun Liang. 2017. "A Deep Reinforcement Learning Framework for the Financial Portfolio Management Problem," 1–31. http://arxiv.org/abs/1706.10059.

Kabbani, Talal, and Engin Duman. 2022. "Deep Reinforcement Learning Approach for Trading Automation in the Stock Market." *IEEE Access* 10: 93564–74. https://doi.org/10.1109/ACCESS.2022.3203697.

Kingma, Diederik P., and Jimmy Ba. 2014. "Adam: A Method for Stochastic Optimization." *arXiv Preprint arXiv:1412.6980*. https://arxiv.org/abs/1412.6980.

Krauss, Christopher, Xuan Anh Do, and Nicolas Huck. 2017. "Deep Neural Networks, Gradient-Boosted Trees, Random Forests: Statistical Arbitrage on the s&p 500." *European Journal of Operational Research* 259 (2): 689–702. https://doi.org/10.1016/j.ejor.2016.10.031.

Kyle, Albert S. 1985. "Continuous Auctions and Insider Trading." *Econometrica* 53 (6):

1315–35. https://doi.org/10.2307/1913210.

Lee, Charles M C, and Mark J Ready. 1991. "Inferring Trade Direction from Intraday Data." *Journal of Finance* 46 (2): 733–46. https://doi.org/10.1111/j.1540-6261.1991.tb02683.x.

Lee, Jae Won. 2001. "Neural Network q-Learning for Stock Trading (Provisional Thesis Citation)."

Liang, Zhipeng, Haoyuan Chen, Jun Zhu, Kunyao Jiang, Yu Li, and Weinan Zhu. 2018. "Adversarial Deep Reinforcement Learning in Portfolio Management." *arXiv Preprint arXiv:1808.09940*. https://arxiv.org/abs/1808.09940.

Lillicrap, Timothy P., Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. 2016. "Continuous Control with Deep Reinforcement Learning." In *International Conference on Learning Representations*.

Lin, Long-Ji. 1992. "Self-Improving Reactive Agents Based on Reinforcement Learning, Planning and Teaching." *Machine Learning* 8 (3–4): 293–321. https://doi.org/10.1007/BF00992699.

López de Prado, Marcos. 2018. *Advances in Financial Machine Learning*. Hoboken, NJ: Wiley.

Majidi, Naseh, Mahdi Shamsi, and Farokh Marvasti. 2024. "Algorithmic Trading Using Continuous Action Space Deep Reinforcement Learning." *Expert Systems with Applications* 237: 121292. https://doi.org/10.1016/j.eswa.2023.121292.

Mnih, Volodymyr, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, et al. 2015. "Human-Level Control Through Deep Reinforcement Learning." *Nature* 518 (7540): 529–33. https://doi.org/10.1038/nature14236.

Moody, John, and Matthew Saffell. 1998. "Reinforcement Learning for Trading."

———. 2001. "Learning to Trade via Direct Reinforcement." *IEEE Transactions on Neural Networks* 12 (4): 875–89. https://doi.org/10.1109/72.935097.

Neuneier, Ralph. 1998. "Enhancing q-Learning for Optimal Asset Allocation." *Advances in Neural Information Processing Systems* 10: 936–42.

Ng, Andrew Y., Daishi Harada, and Stuart Russell. 1999. "Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping." *ICML*, 278–87.

Noonan, Laura. 2017. "JPMorgan to Take AI Trading Global." https://www.ft.com/content/16b8ffb6-7161-11e7-aca6-c6bd07df1a3c?syn-25a6b1a6=1.

Puterman, Martin L. 1994. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. New York: John Wiley & Sons.

Roll, Richard. 1984. "A Simple Implicit Measure of the Effective Bid-Ask Spread in an Efficient Market." *Journal of Finance* 39 (4): 1127–39.

Rummery, Gavin A., and Mahesan Niranjan. 1994. "On-Line q-Learning Using Connectionist Systems." CUED/F-INFENG/TR 166. University of Cambridge, Department of Engineering.

Shen, Yijun. 2021. "Benchmarking Reinforcement Learning Algorithms Across Financial Markets (Provisional Thesis Citation)."

Stoikov, Sasha. 2018. "The Micro-Price: A High-Frequency Estimator of Future Prices." *Quantitative Finance* 18 (12): 1959–66. https://doi.org/10.1080/14697688.2018.1489139.

Sutton, Richard S. 1988. "Learning to Predict by the Methods of Temporal Differences." *Machine Learning* 3 (1): 9–44. https://doi.org/10.1007/BF00115009.

Sutton, Richard S., and Andrew G. Barto. 2018. *Reinforcement Learning: An Introduction*. 2nd ed. Cambridge, MA: MIT Press.

Sutton, Richard S., David McAllester, Satinder Singh, and Yishay Mansour. 2000. "Policy Gradient Methods for Reinforcement Learning with Function Approximation." *Advances in Neural Information Processing Systems* 12: 1057–63.

Tsitsiklis, John N., and Benjamin Van Roy. 1997. "An Analysis of Temporal-Difference Learning with Function Approximation." *IEEE Transactions on Automatic Control* 42 (5): 674–90. https://doi.org/10.1109/9.580874.

Wang, Zhengyao. 2019. "PPO-Based Cryptocurrency Portfolio Management (Provisional Thesis Citation)."

Watkins, Christopher J. C. H. 1989. *Learning from Delayed Rewards*. King's College, University of Cambridge.

Watkins, Christopher J. C. H., and Peter Dayan. 1992. "Q-Learning." *Machine Learning* 8 (3–4): 279–92. https://doi.org/10.1007/BF00992698.

Williams, Ronald J. 1992. "Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning." *Machine Learning* 8 (3–4): 229–56. https://doi.org/10.1007/BF00992696.

Wood, Robert A., Thomas H. McInish, and J. Keith Ord. 1985. "An Investigation of Transactions Data for NYSE Stocks." *Journal of Finance* 40 (3): 723–39.

Xiong, Zhuoran, Xiao-Yang Liu, Shan Zhong, Hongyang Yang, and Anwar Walid. 2018. "Practical Deep Reinforcement Learning Approach for Stock Trading." *arXiv Preprint arXiv:1811.07522*. https://arxiv.org/abs/1811.07522.

Yang, Xiao. 2020. "Deep Reinforcement Learning for High-Frequency Trading with PPO (Provisional Thesis Citation)."

Zhang, Wei. 2020. "Attention-Based Deep Reinforcement Learning for Stock Trading (Provisional Thesis Citation)."

Zhang, Zihao, Stefan Zohren, and Stephen Roberts. 2019. "DeepLOB: Deep Convolutional Neural Networks for Limit Order Books." *IEEE Transactions on Signal Processing* 67 (11): 3001–12. https://doi.org/10.1109/TSP.2019.2907260.

A. Feature Inventory

The tables below catalogue every feature type implemented in the feature registry (`src/trading_rl/features/`). The subset used in the main state-space experiment is marked with [x] in the **In model** column; this subset is defined in `src/configs/features/hft_lob_feature` Feature names match the keys produced by the feature pipeline at runtime. Formulas for the active LOB features are given in Chapter 4.

All features used in the work are z-score normalized using training-split statistics ($\tilde{x}_i = (x_i - \mu_i^{\text{train}})/\sigma_i^{\text{train}}$) with two exceptions noted below the tables: the cyclical time encodings, which are inherently bounded in $[-1, 1]$, and the runtime position feature, which is already bounded in $[-1, 1]$ by the action space definition.

A.1. LOB Microstructure Features

Table 7: LOB microstructure features implemented in the feature registry. The In model column marks features used in the main state-space experiment.

Feature name	Group	Param	In model	Literature
`hft_book_pressure_l0`	Imbalance	level 0	[x]	Cao et al. (2004); Biais et al. (1995)
`hft_book_pressure_l1`	Imbalance	level 1	[x]	Cao et al. (2004)
`hft_book_pressure_l2`	Imbalance	level 2	[x]	Cao et al. (2004)
`hft_order_book_imbalance_3`	Imbalance	$N = 3$	[x]	Cont et al. (2014)
`hft_order_count_imbalance_l0`	Imbalance	level 0	[x]	Biais et al. (1995)
`hft_order_count_imbalance_l1`	Imbalance	level 1		Biais et al. (1995)

Feature name	Group	Param	In model	Literature
`hft_microprice`	Fair value	—	[x]	Stoikov (2018)
`hft_microprice_divergence`	Fair value	—	[x]	Stoikov (2018)
`hft_vwmp_skew`	Fair value	$N = 3$	[x]	Stoikov (2018)
`hft_price_vamp_1k`	Fair value	\$1k		Volume-adjusted mid price
`hft_price_vamp_5k`	Fair value	\$5k		Volume-adjusted mid price
`hft_depth_ratio`	Depth	$N = 4$		Bouchaud et al. (2002)
`hft_spread_bps`	Spread/cost	—	[x]	Kyle (1985); Glosten & Milgrom (1985)
`hft_spread_ratio_50`	Spread/cost	$W = 50$	[x]	Easley et al. (2012)
`hft_spread_ratio_200`	Spread/cost	$W = 200$		Easley et al. (2012)
`hft_bid_convexity`	Spread/cost	bid side	[x]	Bouchaud et al. (2002)
`hft_ask_convexity`	Spread/cost	ask side	[x]	Bouchaud et al. (2002)
`hft_bid_slope`	Spread/cost	5 levels	[x]	Bouchaud et al. (2002); Kyle (1985)
`hft_ask_slope`	Spread/cost	5 levels	[x]	Bouchaud et al. (2002); Kyle (1985)
`hft_ofi`	Order flow	—	[x]	Cont et al. (2014)
`hft_ofi_rolling_10`	Order flow	$W = 10$		Cont et al. (2014)
`hft_ofi_rolling_50`	Order flow	$W = 50$	[x]	Cont et al. (2014)
`hft_ofi_rolling_200`	Order flow	$W = 200$		Cont et al. (2014)
`hft_ofi_multilevel_3l`	Order flow	$N = 3$		Cont et al. (2014)
`hft_ofi_multilevel_5l`	Order flow	$N = 5$		Cont et al. (2014)
`hft_bid_queue_depletion`	Order flow	bid side	[x]	Foucault et al. (2005)
`hft_ask_queue_depletion`	Order flow	ask side	[x]	Foucault et al. (2005)
`hft_signed_trade_flow_50`	Order flow	$W = 50$	[x]	Lee & Ready (1991)

Feature name	Group	Param	In model	Literature
`hft_signed_trade_flow_200`	Order flow	$W = 200$		Lee & Ready (1991)
`hft_odd_lot_trade_ratio_200`	Order flow	$W = 200$	[x]	Boehmer et al. (2021)
`hft_odd_lot_imbalance_200`	Order flow	$W = 200$	[x]	Boehmer et al. (2021)
`hft_vpin_100`	Order flow	$W = 100$		Easley et al. (2012)
`hft_vpin_500`	Order flow	$W = 500$		Easley et al. (2012)
`hft_cancel_to_trade_200`	Order flow	$W = 200$		Foucault et al. (2005)
`hft_cancel_to_trade_1000`	Order flow	$W = 1000$		Foucault et al. (2005)
`hft_trade_arrival_rate_100`	Order flow	$W = 100$		Engle (2000)
`hft_trade_arrival_rate_500`	Order flow	$W = 500$		Engle (2000)
`hft_large_trade_ratio_200`	Order flow	$W = 200$		Institutional flow proxy
`hft_ofi_autocorr_{50,200}`	Order flow	$W \in \{50, 200\}$		Cont et al. (2014)
`hft_inter_event_time`	Regime/temp.	—	[x]	Engle (2000)
`hft_mid_price_acceleration`	Regime/temp.	—	[x]	Abergel et al. (2016)
`hft_hour_sin`	Regime/temp.	—	[x]	Wood et al. (1985)
`hft_hour_cos`	Regime/temp.	—	[x]	Wood et al. (1985)

Literature: the cited work that motivates or introduces the feature; not a claim that the exact implementation follows the cited paper's specification.

A.2. Bar-Level and Technical Features

The features below operate on OHLCV bar data and are not used in the main HFT experiment but are available in the feature registry for medium-frequency trading (MFT) scenarios and ablation studies.

Table 8: Bar-level and technical features available in the feature registry (not used in the main HFT experiment).

Feature name	Group	Param	Notes
`log_return`	Price	—	Log return of close
`high`	Price	—	High / close ratio
`low`	Price	—	Low / close ratio
`simple_return`	Price	—	Simple return
`trend`	Price	—	Not normalized
`rsi`	Price	$P = 14$	Relative strength index
`return_lag_1`	Price	lag 1	Lagged close return
`log_volume`	Volume	—	Log of volume
`volume_change`	Volume	—	Volume change rate
`volume_ma_ratio_20`	Volume	$W = 20$	Volume / MA ratio
`amihud_illiquidity`	Volume	$W = 1$	Amihud (2002)
`relative_volume`	Volume	—	Relative volume ratio
`realized_volatility_20`	Volatility	$W = 20$	Realized vol
`volatility_ratio_5_60`	Volatility	$S = 5, L = 60$	Short / long vol ratio
`hour_sin`	Temporal	—	Not normalized[a]
`hour_cos`	Temporal	—	Not normalized[a]
`day_of_week_sin`	Temporal	—	Not normalized[a]
`day_of_week_cos`	Temporal	—	Not normalized[a]
`minute_of_hour_sin`	Temporal	—	Not normalized[a]
`minute_of_hour_cos`	Temporal	—	Not normalized[a]
`sma_20`	TA-Lib trend	$P = 20$	Simple moving average
`ema_20`	TA-Lib trend	$P = 20$	Exponential moving average
`wma_20`	TA-Lib trend	$P = 20$	Weighted moving average

Feature name	Group	Param	Notes
`dema_20`	TA-Lib trend	$P = 20$	Double EMA
`tema_20`	TA-Lib trend	$P = 20$	Triple EMA
`talib_rsi_14`	TA-Lib momentum	$P = 14$	RSI (TA-Lib)
`macd_12_26_9`	TA-Lib momentum	$f = 12, s = 26, g = 9$	MACD
`mom_10`	TA-Lib momentum	$P = 10$	Momentum
`roc_10`	TA-Lib momentum	$P = 10$	Rate of change
`cmo_14`	TA-Lib momentum	$P = 14$	Chande momentum oscillator
`willr_14`	TA-Lib momentum	$P = 14$	Williams %R
`cci_14`	TA-Lib momentum	$P = 14$	Commodity channel index
`atr_14`	TA-Lib volatility	$P = 14$	Average true range
`natr_14`	TA-Lib volatility	$P = 14$	Normalized ATR
`bbands_20`	TA-Lib volatility	$P = 20$	Bollinger bands
`obv`	TA-Lib volume	—	On-balance volume
`ad`	TA-Lib volume	—	Accumulation / distribution
`adosc_3_10`	TA-Lib volume	$f = 3, s = 10$	AD oscillator
`adx_14`	TA-Lib trend str.	$P = 14$	Average directional index
`aroon_14`	TA-Lib trend str.	$P = 14$	Aroon oscillator

[a] Cyclical time encodings are inherently bounded in $[-1, 1]$ and require no normalization. The runtime position feature (`feature_position` $\in [-1, 1]$) is likewise not normalized; it is already bounded by the action space definition. All other features are z-score normalized on the training split only.

B. TD3 Hyperparameters

The table below lists all hyperparameters for the main AAPL state-space experiment. The full configuration file is `src/configs/scenarios/aapl_td3_hft_lob_state_space.yaml`.

Table 9: TD3 hyperparameters for the main AAPL state-space experiment. Full configuration: `src/configs/scenarios/aapl_td3_hft_lob_state_space.yaml`.

Parameter	Value	Notes
Architecture		
Actor hidden dims	[128, 64]	ReLU activations
Critic hidden dims	[128, 64]	ReLU activations
Optimization		
Actor learning rate	0.0001	Adam
Critic learning rate	0.0001	Adam
Actor weight decay	0.000002	L2 regularization
Critic weight decay	0.000002	
Loss function	Smooth L1 (Huber)	
TD3 mechanisms		
Discount factor γ	0.995	
Target update τ	0.001	Soft update
Policy delay	2	Critic updates per actor update
Target policy noise σ	0.2	Added to target action
Noise clip	0.3	
Exploration noise std	0.1	During training only
Training		
Total steps	2,000,000	
Initial random steps	1,000	Warm-up before learning
Frames per batch	200	
Optimisation steps per batch	5	
Mini-batch size	128	

Parameter	Value	Notes
Replay buffer size	100,000	
Data		
Training rows	15,000	Chronological split
Reward type	Differential Sharpe	$\eta = 0.01$
Transaction fee	0.000	Baseline setting
Random seed	42	

C. Experiment Configuration Example

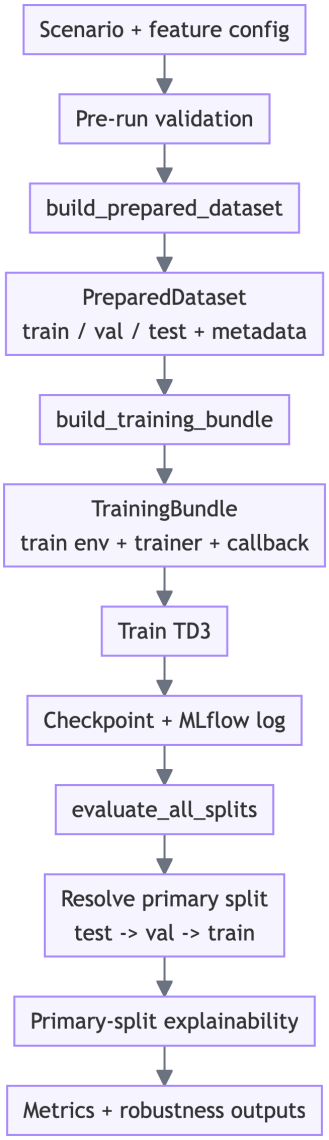
The YAML snippet below is a condensed version of the main experiment configuration, illustrating the configuration-driven workflow described in Chapter 5.

```yaml
seed: 42

data:
  data_path: "./data/raw/stocks/AAPL_2026-02-25_2026-02-27_raw_mbp-10_us_hours.parquet"
  feature_config: "src/configs/features/hft_lob_features_state_space.yaml"
  train_size: 15000

env:
  backend: "tradingenv"
  mode: "hft"
  trading_fees: 0.000
  price_column: "close"
  reward_type: "differential_sharpe"
  reward_eta: 0.01

network:
  actor_hidden_dims: [128, 64]
```

```yaml
  value_hidden_dims: [128, 64]


training:
  algorithm: "TD3"
  actor_lr: 0.0001
  value_lr: 0.0001
  gamma: 0.995
  tau: 0.001
  policy_noise: 0.2
  noise_clip: 0.3
  policy_delay: 2
  exploration_noise_std: 0.1
  max_steps: 2000000
  sample_size: 128
  buffer_size: 100000
  loss_function: "smooth_l1"
```

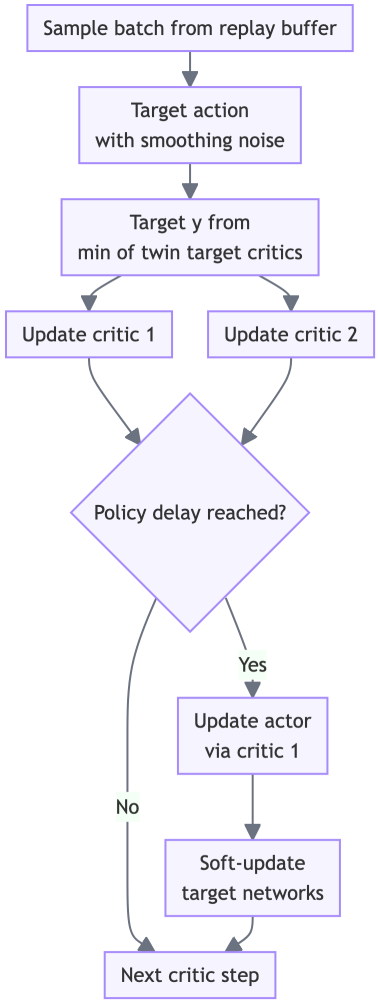
D. TD3 Training Workflow Diagram

The diagram below summarizes the configuration-driven workflow described in Chapter 5.



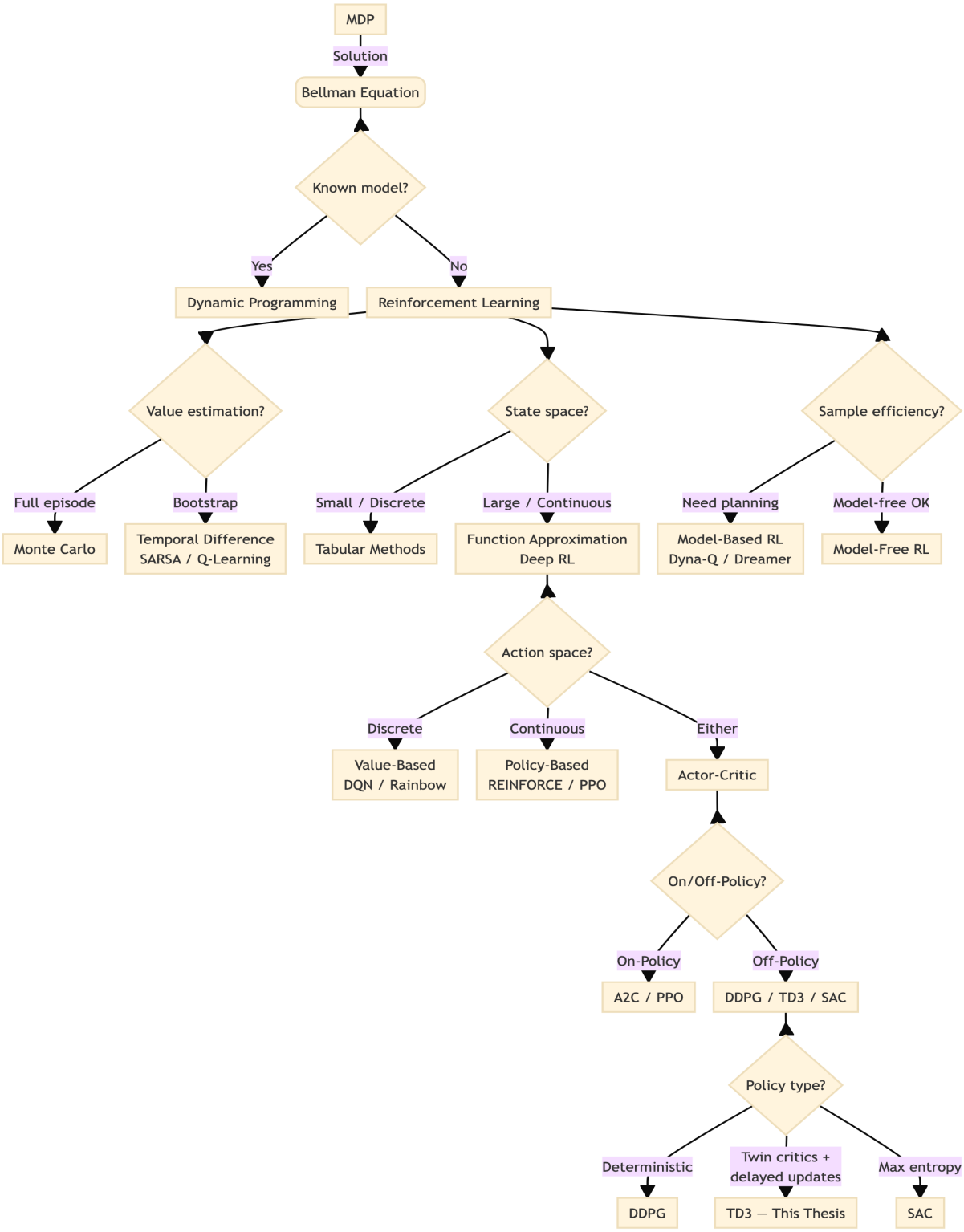
E. TD3 Update Mechanics Diagram

The diagram below summarizes the core TD3 update mechanics used in the thesis implementation, abstracting away from project-specific plumbing.



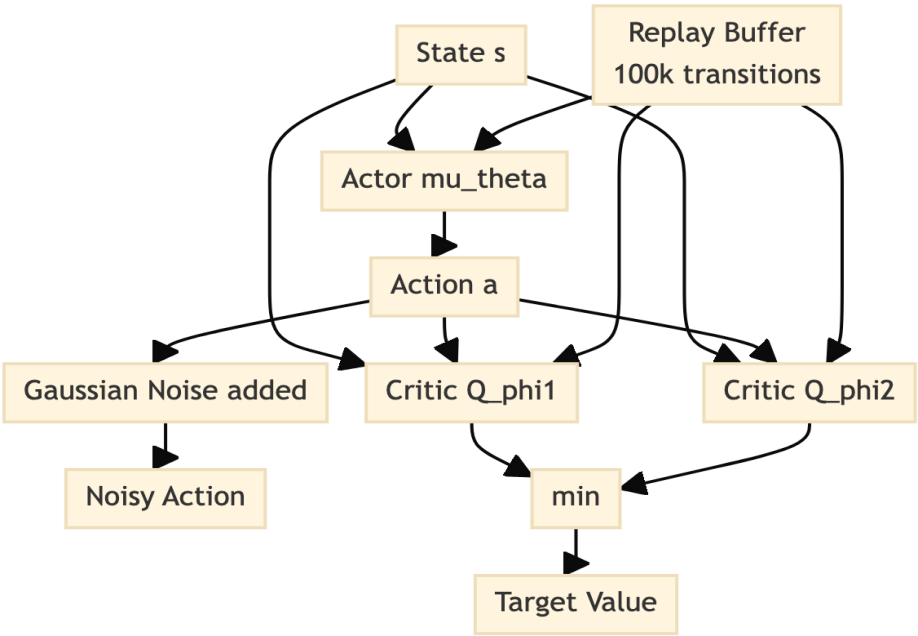
F. RL Algorithm Taxonomy

The diagram below maps the reinforcement learning landscape from the Bellman equation through the key algorithmic relaxations, showing where TD3 sits relative to other families of methods. Source: author's own diagram, based on Sutton and Barto (2018).


MDP
Solution
Bellman Equation
Known model?
Yes
No
Dynamic Programming
Reinforcement Learning
Value estimation?
State space?
Sample efficiency?
Full episode
Bootstrap
Small / Discrete
Large / Continuous
Need planning
Model-free OK
Monte Carlo
Temporal Difference
SARSA / Q-Learning
Tabular Methods
Function Approximation
Deep RL
Model-Based RL
Dyna-Q / Dreamer
Model-Free RL
Action space?
Discrete
Continuous
Either
Value-Based
DQN / Rainbow
Policy-Based
REINFORCE / PPO
Actor-Critic
On/Off-Policy?
On-Policy
Off-Policy
A2C / PPO
DDPG / TD3 / SAC
Policy type?
Deterministic
Twin critics +
delayed updates
Max entropy
DDPG
TD3 — This Thesis
SAC

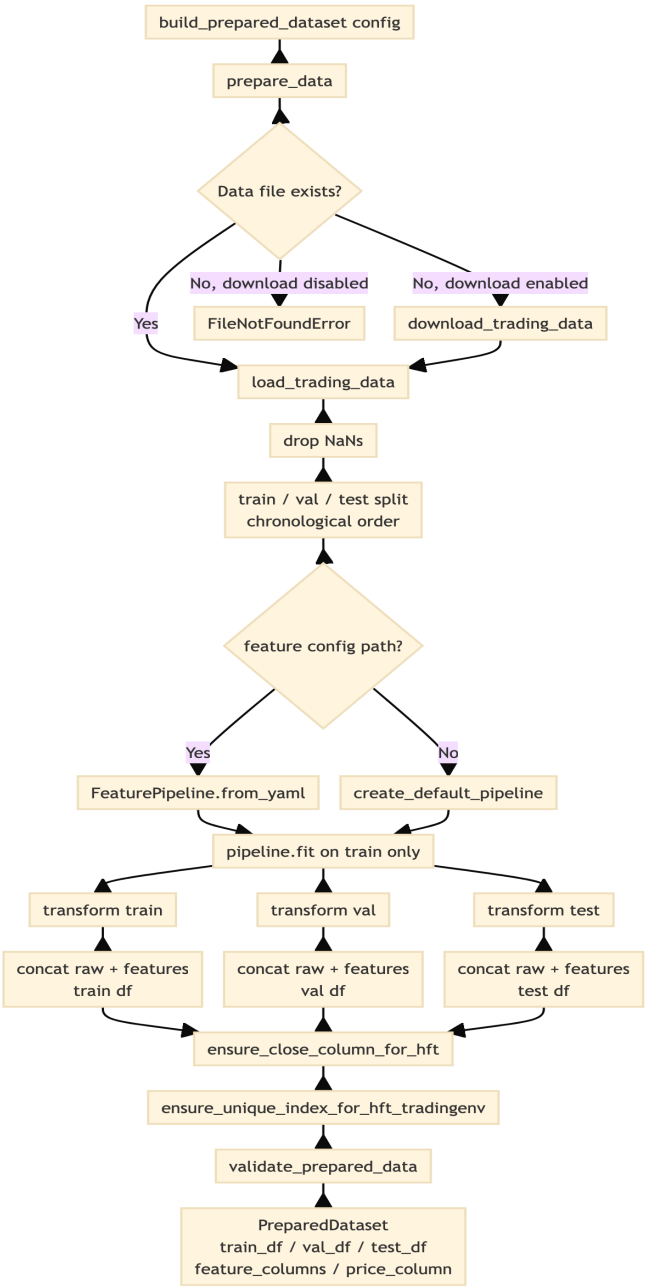
G. TD3 Network Architecture

The diagram below shows how the actor, twin critics, and replay buffer interact in the TD3 architecture used here.



H. Data Preparation Pipeline

The diagram below shows the feature pipeline design that ensures no information from the validation or test splits leaks into the fitted normalization parameters.



Glossary of Abbreviations and Key Terms

This glossary defines abbreviations and domain-specific terms used throughout the thesis. Terms are grouped by domain and listed alphabetically within each group.

H.0.1. Market Microstructure and Trading

Term	Definition
Adverse selection	The risk that a counterparty in a trade possesses superior private information, causing uninformed participants (e.g., market makers) to trade at a disadvantage.
Ask slope	Price elasticity of the ask side of the order book across multiple depth levels; a proxy for market impact per unit of sell-side volume.
Basis points (bps)	One hundredth of a percentage point (0.01%). Used to express spreads and returns at fine granularity.
Bid-ask spread	The difference between the lowest ask price and the highest bid price in the order book; the primary observable measure of transaction cost and adverse selection risk.
Bid slope	Price elasticity of the bid side of the order book across multiple depth levels; a proxy for market impact per unit of buy-side volume.
Book convexity	A measure of curvature in the price schedule of the order book, capturing non-linear spacing between price levels on each side.
CumDelta (Cumulative Delta)	Signed trade flow aggregated over a rolling window; the net difference between buyer-initiated and seller-initiated executed volume.
Depth imbalance	Asymmetry in resting volume between the bid and ask sides of the order book at one or more price levels.
ETF	Exchange-Traded Fund. A publicly traded fund that tracks an index, sector, or asset class.
HFT	High-Frequency Trading. Algorithmic trading operating at tick or sub-second timescales, relying on order-book data and microstructure signals rather than price bars.
Inter-event time	The elapsed time between consecutive limit order book updates; a proxy for market activity intensity.

Term	Definition
Kyle's lambda (λ)	A measure of price impact introduced by Kyle (1985): the sensitivity of price changes to net order flow. Higher λ implies lower market liquidity.
LOB	Limit Order Book. The electronic record of all outstanding buy (bid) and sell (ask) limit orders at each price level, maintained by an exchange.
Market maker	A liquidity provider that continuously posts limit orders on both sides of the book, profiting from the bid-ask spread while bearing inventory and adverse selection risk.
MFT	Medium-Frequency Trading. Trading strategies operating at intraday bar (minute to hour) timescales, as distinct from high-frequency tick-level strategies.
Microprice	A volume-weighted estimate of the next transaction price, derived from top-of-book bid and ask quantities (Stoikov 2018). Adjusts the mid-price for queue imbalance.
MicroDiv (Microprice Divergence)	The difference between the microprice and the mid-price; a stationary signal capturing the directional imbalance embedded in the top-of-book queues.
Mid-price	The arithmetic average of the best bid and best ask prices; commonly used as a reference fair value when the spread is symmetric.
OBI (Order Book Imbalance)	Normalized difference in resting volume between bid and ask sides, aggregated across multiple depth levels.
OCI (Order Count Imbalance)	Normalized difference in the number of resting orders between bid and ask sides at a given price level; sensitive to order fragmentation masked by volume-based measures.
OFI (Order Flow Imbalance)	Net directional pressure exerted on the best bid and ask queues by limit order arrivals, cancellations, and executions (Cont, Kukanov, and Stoikov 2014). The empirically dominant contemporaneous predictor of short-term price changes.

Term	Definition
OHLCV	Open, High, Low, Close, Volume. The standard aggregated price bar representation used in low- and medium-frequency trading; does not capture intra-bar queue dynamics.
Odd-lot	A trade smaller than the standard round lot size (typically below 100 shares). Predominantly reflects retail and algorithmic small-lot activity.
Queue depletion	The rate at which the best-bid or best-ask queue is consumed by incoming market orders across consecutive snapshots; a leading indicator of imminent price movement.
RSI	Relative Strength Index. A momentum oscillator measuring the speed and magnitude of price changes; used in rule-based and technical trading strategies.
Spread ratio	The current bid-ask spread divided by its recent rolling mean; a regime signal separating absolute spread level from elevated adverse selection conditions.
VIX	CBOE Volatility Index. A measure of expected 30-day S&P 500 volatility derived from options prices; a common proxy for broad market fear or uncertainty.
VWAP	Volume-Weighted Average Price. The average price of an asset weighted by traded volume over a period; a common execution benchmark.

H.0.2. Reinforcement Learning and Machine Learning

Term	Definition
Actor-critic	A class of reinforcement learning architectures combining a policy network (actor) that selects actions with a value network (critic) that evaluates them. Reduces gradient variance relative to pure policy gradient methods.

Term	Definition
DDPG	Deep Deterministic Policy Gradient (Lillicrap et al. 2016). An off-policy actor-critic algorithm for continuous action spaces using deterministic policies and experience replay. Predecessor to TD3.
DQN	Deep Q-Network (Mnih et al. 2015). A value-based deep RL algorithm combining Q-learning with a deep neural network, experience replay, and a frozen target network. Restricted to discrete action spaces.
DSR	Differential Sharpe Ratio (Moody and Saffell 1998). A reward formulation that approximates the gradient of the Sharpe Ratio with respect to returns, aligning the optimization objective with risk-adjusted performance.
Exploration-exploitation tradeoff	The tension in reinforcement learning between taking actions that yield known rewards (exploitation) and trying new actions to discover potentially better outcomes (exploration). Particularly costly in trading due to transaction costs.
MDP	Markov Decision Process. The formal framework for sequential decision-making under uncertainty, consisting of states, actions, a transition function, and a reward function. Assumes that future states depend only on the current state and action, not on history.
Off-policy	A class of RL algorithms that learn a target policy from data collected by a different behavior policy. Enables experience replay and reuse of historical data. TD3 is off-policy.
On-policy	A class of RL algorithms that must learn from data collected by the same policy being improved. More stable but less sample-efficient than off-policy methods.

Term	Definition
PCA	Principal Component Analysis. A dimensionality reduction technique that projects data onto orthogonal axes of maximum variance.
Policy gradient	A family of RL algorithms that directly optimize the policy parameters through gradient ascent on expected cumulative reward. Handles continuous action spaces naturally.
PPO	Proximal Policy Optimization. An on-policy actor-critic algorithm that constrains policy updates to a trust region to improve stability.
Q-learning	A model-free, off-policy value-based RL algorithm that learns the action-value function $Q(s, a)$ through temporal difference updates (Watkins and Dayan 1992).
Replay buffer	A data structure storing past environment transitions (s_t, a_t, r_t, s_{t+1}) for reuse in off-policy training. Decorrelates training samples and improves sample efficiency.
REINFORCE	A Monte Carlo policy gradient algorithm (Williams 1992) that updates policy parameters using full-episode return estimates. High variance; foundational to modern policy gradient methods.
RL	Reinforcement Learning. A machine learning paradigm in which an agent learns to make sequential decisions by interacting with an environment and maximizing cumulative reward.
SAC	Soft Actor-Critic (Haarnoja et al. 2018). An off-policy actor-critic algorithm incorporating a maximum-entropy objective to encourage exploration through stochastic policies.
SARSA	State-Action-Reward-State-Action. An on-policy temporal difference algorithm that updates $Q(s, a)$ using the action actually taken by the current policy.

Term	Definition
TD3	Twin Delayed Deep Deterministic Policy Gradient (Fujimoto, Hoof, and Meger 2018). An off-policy actor-critic algorithm for continuous control that extends DDPG with clipped double Q-learning, delayed policy updates, and target policy smoothing to reduce overestimation bias. The primary algorithm used in this thesis.
Temporal difference (TD) learning	A class of RL methods that update value estimates based on the difference between successive predictions, without requiring full episode trajectories.
Value function	A function estimating the expected cumulative discounted reward from a given state (or state-action pair) under a given policy. The critic network in actor-critic algorithms approximates this function.

H.0.3. Other

Term	Definition
AAPL	The NASDAQ ticker symbol for Apple Inc. The equity used as the single-asset trading instrument throughout this thesis.
Sharpe ratio	A risk-adjusted performance measure defined as the ratio of mean excess return to return volatility. Higher values indicate better return per unit of risk.